

Bachelorarbeit

Portierung und Erweiterung eines Plasma-Systems zur einfachen Nutzung weiterer Hardware-Komponenten

zur Erlangung des akademischen Grades

Bachelor of Engineering (B.Eng.)

vorgelegt dem
Fachbereich Informationstechnik, Elektrotechnik und Mechatronik
der Technischen Hochschule Mittelhessen
von

Sergej Zuyev
geb. in Lugansk

TODO: Datum

Referent der Arbeit:	Prof. Dr. Hartmut Weber
Korreferent der Arbeit:	Prof. Dr.-Ing. Martin Gräfe

In diesem Dokument werden die Portierung eines Plasma (MIPS-I) Systems auf ein Field Programmable Gate Array (FPGA) des Herstellers Altera/Intel und Erweiterung um zusätzliche Komponenten, , beschrieben. Besonderes Augenmerk wird auf die verwendeten Bussysteme, die Anbindung an einen Advanced RISC Machine (ARM)-Prozessor und die Treiberentwicklung für Linux gelegt.

In this document the porting of a Plasma (MIPS-I) System to Altera/Intel FPGAs and extension with additional components to ease the usage of further hardware components is described. The emphasis lies on used bus-systems, the connection to an ARM-Processors and driver development for Linux.

die eine einfachere Nutzung weiterer Hardware-Komponenten erlauben

Eidesstattliche Erklärung

Eidesstattliche Erklärung zur <-Arbeit>

an THM Richtlinien anpassen

Ich versichere, die von mir vorgelegte Arbeit selbstständig verfasst zu haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Arbeiten anderer entnommen sind, habe ich als entnommen kenntlich gemacht. Sämtliche Quellen und Hilfsmittel, die ich für die Arbeit benutzt habe, sind angegeben. Die Arbeit hat mit gleichem Inhalt bzw. in wesentlichen Teilen noch keiner anderen Prüfungsbehörde vorgelegen.

$Unterschrift:$ $Ort, Datum:$

Inhaltsverzeichnis

Glossar **IV**

Abbildungsverzeichnis **VI**

Tabellenverzeichnis **VIII**

Quellcodeverzeichnis **IX**

1. Einleitung **1**
1.1. Motivation 1
1.2. Problemstellung 1
1.3. Anforderungen 2
1.4. Organisation dieser Arbeit 2

2. Grundlagen **3**
2.1. System-On-Chip 3
2.2. Bussysteme 4
2.2.1. Topologien 4
2.2.2. Wishbone 6
2.2.3. Avalon 8
2.3. Plasma CPU 9
2.4. ZedBoard development kit 10
2.5. DE1-SoC development kit 13
2.6. Entwicklungsworkflow mit Quartus 15
2.7. Devicetree 15
2.8. Linux 15

3. Systemarchitektur **21**
3.1. Plasma 21
3.2. Plasma-SoC auf dem ZedBoard 21
3.3. Plasma auf dem DE1-SoC-Board 23

4. Umsetzung **25**
4.1. Entwicklung des Wishbone-basierten Systems 25

4.2. Entwicklung der Plasma-Firmware . . . 31
4.2.1. Treiber . . . 31
4.2.2. Dynamisches Speichermanagement . . . 35
4.2.3. I/O . . . 35
4.3. Portierung der Hardware für DE1-SoC . . . 35
4.3.1. RAM . . . 36
4.3.2. Register-File . . . 38
4.3.3. Laufzeitverhalten . . . 40
4.4. Erweiterung der Hardware für DE1-SoC . . . 40
4.4.1. Avalon-Wishbone Bridge . . . 40
4.4.2. Arbiter . . . 42
4.4.3. Wishbone-Bus . . . 45
4.4.4. Externer SDRAM . . . 46
4.5. Erweiterung der Hilfssoftware . . . 48
4.6. Portierung der Plasma-Firmware und der Beispielanwendung . . . 49
4.7. Einrichtung des Linux-Betriebssystems . . . 50
4.7.1. Preloader . . . 50
4.7.2. U-Boot . . . 50
4.7.3. Kernel . . . 50
4.7.4. Devicetree . . . 51
4.7.5. Systemabbild . . . 55
4.8. Entwicklung von Linux-Anwendungen für das HPS . . . 56
4.9. Entwicklung von Linux-Treibern für das HPS . . . 56

5. Ergebnisse und Ausblick **58**
5.1. Zusammenfassung . . . 58
5.2. Ausblick . . . 58

A. Übersicht über elektronische Anhänge **61**

Glossar

ARM Advanced RISC Machine. 2, 4, 10, 13, 25, B

Bridge Eine Komponente, die eine Kommunikation zwischen zwei oder mehr Bussystemen untereinander ermöglicht. 40

CPU Central Processing Unit. 3, 4, 9, 10, 25, 45

CSR Control/Status Register. 27

DDR SDRAM Double Data Rate Synchronous Dynamic Random Access Memory. 9, 10

DMA Direct Memory Access. 1

DSP Digital Signal Processor. 3

Firmware In elektronischen Geräten eingebettete Software. 31

FPGA Field Programmable Gate Array. 1, 2, 10, 13, 15, 21, 23, 36, 40, 46, 47, B

GPIO General-Purpose Input/Output. 21, 27, 29, 54

GPU Graphics Processing Unit. 3

GSM Global System for Mobile Communications, früher Groupe Spécial Mobile: Mobilfunkstandard für volldigitale Mobilfunknetze. 3

HDL Hardware Definition Language. 3

HDMI High-Definition Multimedia Interface. 10

HPS Hard Processor System. 14, 25

Inferring Fähigkeit des HDL-Compilers, aus Sprachkonstrukten Strukturelemente wie z.B RAM oder FIFO zu generieren. 37, 40

Interconnect Kommunikationsschnittstelle zwischen mehreren Geräten. 21, 25, 40

IoT Internet of Things. 3

IRQ Interrupt Request. 27, 28, 34, VIII

LPM library of parameterized modules. 37, 38, 40

LSB least significant bit. 42

MIO Multiplexed I/O. 10

MIPS Microprocessor without Interlocked Pipeline Stages. 10

PL Programmable Logic. 10

PLL phase locked loop. 47

PMOD Peripheral Module. 10

PS Processing System. 10

RAM Random Access Memory. 1, 21, 23, 25–27, 36–38, 40, 41

RISC Reduced Instruction Set Computer. 9

SoC System-on-Chip. 1, 3, 4, 6, 10, 13, 25, 31, 40, 55, 56

SOPC System-on-a-programmable-chip. 8

SPI Serial Peripheral Interface bus. 21, 27

SRAM Static Random Access Memory. 9

UART Universal Asynchronous Receiver-Transmitter. 9, 10, 21, 23, 26, 27, 29, 32, 35, 49

USB Universal Serial Bus. 10

VGA Video Graphics Array. 10

Abbildungsverzeichnis

2.1. Crossbar (Switch) [7, S. 17] 5
2.2. Data flow [7, S. 17] 5
2.3. Point to point [7, S. 21] 6
2.4. Shared bus [7, S. 21] 6
2.5. Standard connection for timing diagrams [7, S. 14] 7
2.6. Avalon-Bus basiertes System [6, S. 7–6] 8
2.7. Plasma-CPU Blockdiagramm [1] 11
2.8. ZedBoard Blockdiagramm [3, S. 3] 12
2.9. DE1-SoC Blockdiagramm [4, S. 9] 13
2.10. DE1-SoC Oberseite [4, S. 7] 15
2.11. Quartus Hauptprogramm 16
2.12. Quartus Platform Designer 17
2.13. Quartus In-System Memory Content Editor 18
2.14. Signal Tap Logic Analyzer 19
2.15. Mentor Graphics Modelsim 20

3.1. Blockdiagramm des Plasma im Ausgangszustand 22
3.2. Blockdiagramm des Plasma-SoC auf dem ZedBoard 22
3.3. Blockdiagramm des Plasma-SoC auf dem DE1-SoC 23
3.4. Blockdiagramm des Plasma-SoC auf dem DE1-SoC mit Verbindung zu weiteren IP-Cores und ARM Prozessor über den Avalon-Bus 24

4.1. DE1-SoC SystemBuilder 36
4.2. Quartus Boardeinstellungen 37
4.3. Ausgelesener Speicherinhalt 38
4.4. Vergleich der RTL-Netzlisten 39
4.5. Avalon→Wishbone-Bridge Timings 43
4.6. Wishbone→Avalon-Bridge Timings 44
4.7. Parametrisierung des SDRAM-Controllers 47
4.8. Parametrisierung des *Clocks Signals for DE-Series Board Peripherals*-IP-Core 48
4.9. Mit SignalTap aufgezeichneter Zugriff auf den externen SDRAM . . . 49
4.10. Ausschnitt aus der Kernel-Konfiguration 51

4.11. Ubuntu 12.04-Distribution mit angepasstem Linux 4.9.78 Kernel . . . 56

Tabellenverzeichnis

2.1. Avalon-MM Signale [8, S. 14–16] 9

4.1. Adresszuordnung 26
4.2. Allgemeine Register-Map 27
4.3. Interrupt Controller Register-Map 28
4.4. Interrupt Requests (IRQs) 28
4.5. Zähler Register-Map 28
4.6. Timer Register-Map 29
4.7. Timer Zeiteinheiten Kodierung und Reloadwert [ticks] 29
4.8. UART-Controller Register-Map 29
4.9. GPIO-Controller Register-Map 30
4.10. SPI Controller Register-Map 30
4.11. Display-Controller Register-Map 30
4.12. Control Register Bits 31
4.13. Allgemeine Treiberfunktionen 33
4.14. Treiberfunktionen des Display-Controllers 33
4.15. Treiberfunktionen des Timers 33
4.16. Treiberfunktionen des Zählers 34
4.17. Treiberfunktionen des Display-Controllers 34
4.18. Treiberfunktionen des Interrupt-Controllers 34
4.19. Plasma-SoC Adresszuordnung 41
4.20. Avalon-MM und Wishbone Schnittstellen 42
4.21. Timing Parameter [19, S. 1] 46
4.22. Adresstabelle [19, S. 1] 46

Quellcodeverzeichnis

4.1. Boot-Script 50
4.2. Ausschnitt aus der generierten DTS-Datei 51
4.3. Ausschnitt aus der generierten DTS-Datei 52

1. Einleitung

Diese Bachelorarbeit befasst sich mit der Portierung und Erweiterung eines „Plasma“-Systems [1] von FPGAs des Herstellers Xilinx auf FPGAs des Herstellers Altera/Intel. Betrachtet werden hierbei sowohl die Hard- als auch Softwareaspekte.

Dieses Kapitel dient der Einführung in die Bachelorarbeit und die Problemstellung.

1.1. Motivation

Langfristig gesehen soll es möglich sein, einen hardware-basierten Beschleuniger zur Bildklassifizierung mittels binarisierter neuronaler Netze[2] in das zugrundeliegende „Plasma“-System zu integrieren. Bevor dies geschehen kann, muss das „Plasma“-System auf eine mit der Altera-OpenCL-Toolchain kompatible Hardware-Basis portiert und erweitert werden.

1.2. Problemstellung

Derzeit ist das Plasma-System-on-Chip (SoC) nur auf dem Digilent Zedboard[1] mit einem Zynq-7000 FPGA lauffähig. Ziel der Arbeit ist es, eine Portierung für das Terasic DE1-SoC[2] Board mit einem Cyclone V FPGA vorzunehmen, und es soweit zu erweitern, dass eine einfache Kommunikation mit den Komponenten und Peripherie wie z.B Direct Memory Access (DMA)-Controller und externem Random Access Memory (RAM) möglich wird. Dafür soll ebenso die Software-Basis in Form von Firmware, Bertriebssystem und Hilfs-Software angepasst und erweitert werden.

[1] `http://zedboard.org/product/zedboard`
[2] `https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&No=836`

1.3. Anforderungen

Das Plasma-System soll auf dem FPGA des DE1-SoC-Board lauffähig sein und mit einem ARM Prozessor kommunizieren können. Außerdem soll ein auf dem DE1-SoC-Board auszuführendes Linux-Betriebssystem eingerichtet werden, welches auf das Plasma-System zugreifen können soll. Um die Funktionsfähigkeit der Lösung zu beweisen, sollen Beispielanwendungen entwickelt und auf dem Plasma-System und der ARM-CPU unter Linux ausgeführt werden. Zusätzlich soll ein für das Linux-Betriebssystem zu entwickelnder Treiber lauffähig sein.

1.4. Organisation dieser Arbeit

Zunächst werden in Kapitel 2 die für ein besseres Verständnis dieser Bachelorarbeit notwendigen Grundlagen erklärt.

Darauffolgend werden in Kapitel 3 die Systemarchitekturen vorgestellt, die im Verlauf dieser Bachelorarbeit umgesetzt werden.

In Kapitel 4 wird die Umsetzung in Hardware und Software sowie die Erstellung eines lauffähigen Linux-Systems im Detail behandelt.

Die in vorangehenden Kapiteln gesammelten Erkenntnisse werden in Kapitel 5 zusammengefasst und und ein Ausblick auf zukünftige Entwicklung vorgestellt.

2. Grundlagen

In diesem Kapitel wird zunächst auf die Motivation für ein SoC und dessen Grundaufbau eingegangen. In diesem Zusammenhang wird auf die Bus-Architekturen „Wishbone“ und „Avalon“ eingegangen. Danach wird das Plasma-System von Steve Rhoads, sowie die Entwicklungs- und Evaluationsboards ZedBoard [3] von Digilent und DE1-SoC [4] von Terasic vorgestellt

2.1. System-On-Chip

Der Begriff SoC wurde durch die Integration eines gesamten Systems (Prozessor, Speicher und Peripherie) auf einem einzigen Chip geprägt. Die SoCs sind aus modernen Geräten nicht wegzudenken. Ein prägnantes Beispiel hierfür ist ein Smartphone: auf einem einzigen Chip sind beispielsweise eine Central Processing Unit (CPU), Graphics Processing Unit (GPU), GSM-Modem, ein Display-Controller sowie der Kamera-Controller samt einem Digital Signal Processor (DSP) zur Bildverarbeitung integriert. Moderne Medizin-, Automobil-, Avionik-, Raumfahrt- und Militär-Systeme wären ohne SoCs nicht möglich. Doch auch banal erscheinende Geräte wie Küchengeräte und sogar die Heimbeleuchtung (Internet of Things (IoT) und Smart-Home) profitieren von der zunehmenden Miniaturisierung und den Einsatzmöglichkeiten der SoCs.

Der Entwurf eines SoC ist mit einem sehr hohen Zeit- und Arbeitsaufwand verbunden. Je komplexer das System, desto aufwändiger wird ein von Grund auf neuer Aufbau. [5, S. 189] Darum wird heutzutage, analog zur Softwareentwicklung, auf erprobte, wiederverwendbare Komponenten gesetzt. Diese sind als IP-Cores verfügbar und können in eigene Projekte integriert werden. Davon sind einige parametrisierbar und somit auf Projektbedürfnisse anpassbar. Ein IP-Core kann als high-level Hardware Definition Language (HDL) Code, auch Soft-Core genannt, oder auch als ein detailliertes Layout auf Transistorebene, Hard-Core genannt, ausgeliefert werden. [5, S. 189]

Ein gutes Beispiel für parametrisierbare IP-Cores ist die ARM-CPU. ARM als Unternehmen ist kein Prozessorhersteller an sich, sondern bietet ledeglich die CPU-Architektur als IP-Core zur Erwerb an. Ein Interessent kann diese lizenzieren, auf seine Bedürfnisse anpassen und bei einem Halbleiterfertiger in Produktion geben. Durch Kombination verschiedener auf dem Markt verfügbarer IP-Cores kann innerhalb kürzester Zeit ein neues Produkt mit einem gegenüber den Einzelkomponenten erhöhten Mehrwert entwickelt werden.

2.2. Bussysteme

In vorigen Abschnitten wurden beispielhaft einige Komponenten eines SoC genannt. Im Wesentlichen handelt es sich hierbei um eine CPU, die mit sämtlichen domänenspezifischen Prozessoren und Peripheriegeräten bzw. Schnittstellen kommuniziert. Essentiell hierfür ist ein Bus-System, welches allen relevanten Modulen ein Kommunikationsmedium bereitstellt. Verbreitet sind standartisierte Bus-Architekturen wie AMBA, CoreConnect, Wishbone, STBus und weitere. [6, S. 2] AMBA ist sehr verbreitet in der Entwicklung von SoCs auf Basis von ARM-Prozessoren. In dieser Arbeit wird besondere Aufmerksamkeit dem Wishbone- und Avalon-Bus gewidmet.

2.2.1. Topologien

Unter einer Bus-Topologie versteht man die Anordnung der Master- und Slavegeräte zueinander. Je nach Topologie muss u.U. eine Arbitrierung vorgenommen werden, um Ressourcen unter mehreren Teilnehmern verteilen zu können. Die nachfolgenden Abbildungen entstammen der Wishbone-Spezifikation [7] und sind auch auf andere Bussysteme übertragbar.

In Abb. 2.1 ist eine Crossbar-Switch-Topologie dargestellt. In diesem Fall können mehrere Master zeitgleich eine Verbindung zu einem Slave aufnehmen. Jede Verbindung ist unabhängiig von den anderen und kann parallel betrieben werden. Durch diese Parallelität erhöht sich der Durchsatz des ganzen Systems. Diese Topologie ist geschwindigkeitsmäßig den klassischen Bussystemen überlegen. Grundsätzlich unterstützen die Crossbar Routingmechanismen dynamische Konfiguration, was zu einen zuverlässigem und konfigurierbaren Netzwerk führt. Außerdem sind die meisten Crossbar-Architekturen skalierbar, können also bei Bedarf erweitert werden. [7, S. 16]

In Abb. 2.2 ist eine Data-Flow-Topologie dargestellt. Hierbei fließt der Datenstrom sequentiell durch einen vordefinierten Satz an IP-Cores. Data-Flow-Architekturen

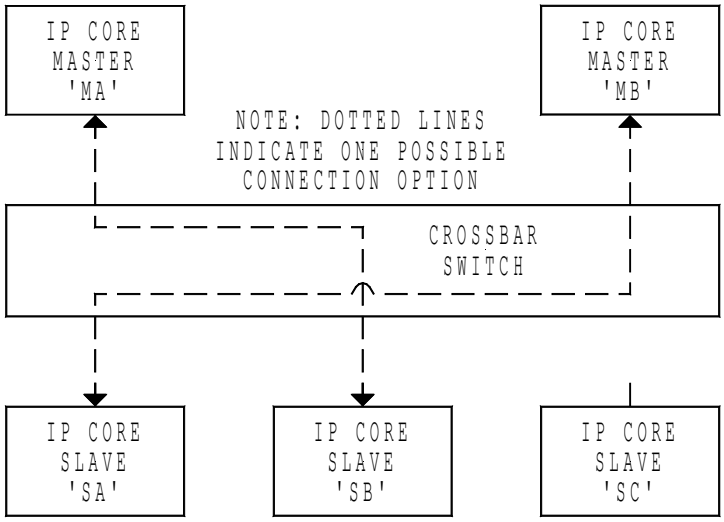


Abbildung 2.1.: Crossbar (Switch) [7, S. 17]

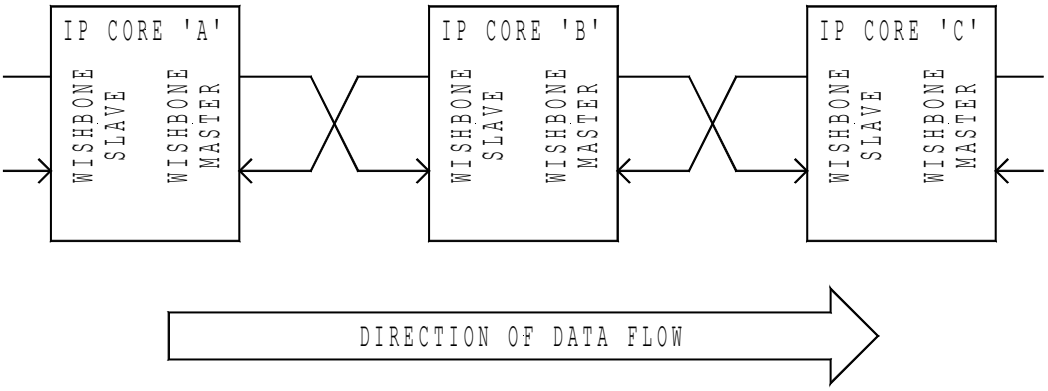


Abbildung 2.2.: Data flow [7, S. 17]

weisen oft eine Parallelität auf: eine oder mehrere werden Funktionen zeitgleich ausgeführt. [7, S. 17]

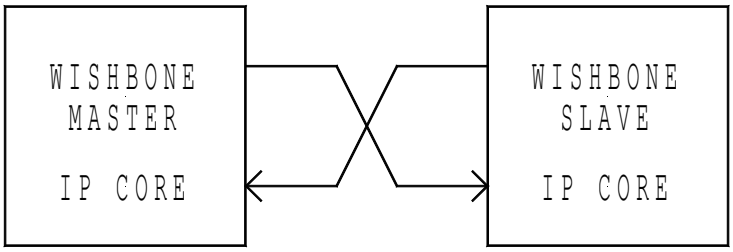


Abbildung 2.3.: Point to point [7, S. 21]

Eine Punkt-zu-Punkt Topologie, wie in Abb. 2.3 abgebildet, ist die einfachste aller Topologien mit einem einzigen Master und einem Slave.[7, S. 20]

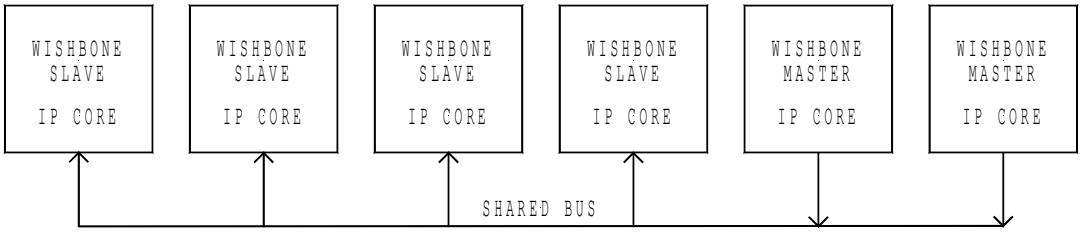


Abbildung 2.4.: Shared bus [7, S. 21]

In Abb. 2.4 ist eine Shared-Bus-Topologie dargestellt. Hierbei initiiert ein Master einen Buszyklus zum Slave. Traditionelle Busse wie VMEbus und PCI Bus nutzen diese Topologie. Architekturbedingt kann hier nur ein Master zu einem Zeitpunkt den Bus nutzen. [7, S. 21]

2.2.2. Wishbone

Wishbone ist ein Open-Source-Hardware Computerbus, welcher die Kommunikation der Komponenten eines SoC durch eine einheitliche Schnittstelle deutlich vereinfacht. Entwickelt wurde dieser von der Silicore Corporation, und später von OpenCores[1] veröffentlicht. Überwiegend wird Wishbone-Standard von quelloffenen Hardware-Projekten verwendet.

Eine mögliche Portkonfiguration für Wishbone-Master bzw Slave ist in Abb. 2.5 dargestellt.

[1] `https://opencores.org`

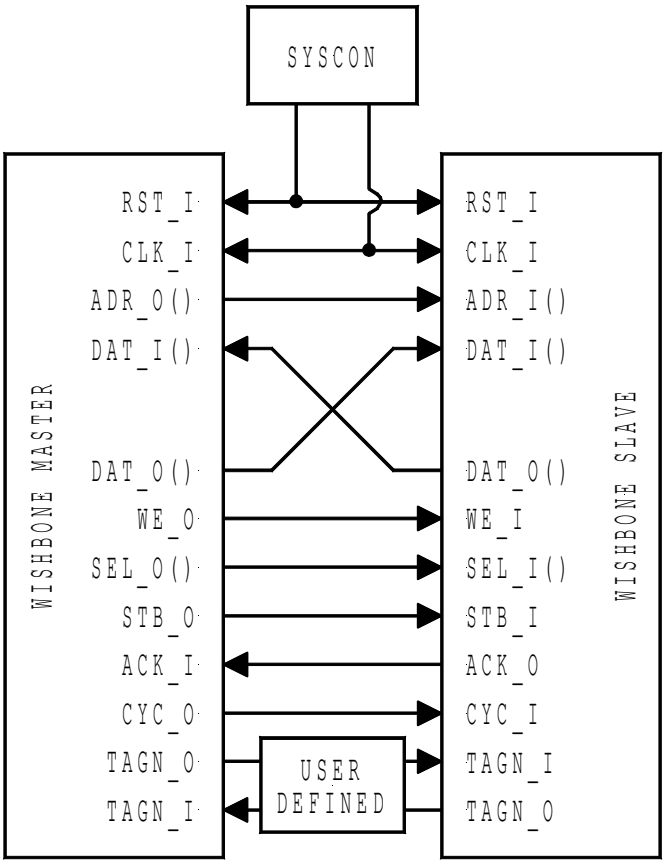


Abbildung 2.5.: Standard connection for timing diagrams [7, S. 14]

2.2.3. Avalon

Avalon ist eine proprietäre Bus-Architektur und wurde ursprünglich von Altera entwickelt, um On-Chip-Prozessoren und Peripherie zu einem System-on-a-programmable-chip (SOPC) zu verknüpfen. [6, S. 7–6] Nutzbar ist Avalon als eine Sammlung parametrisierbarer IP-Cores, die vom Platform Designer (früher als Qsys bzw. SOPC Builder bekannt) im Laufe der Systementwicklung automatisch generiert und parametrisert werden.

Ein mögliches System mit Avalon ist auf Abb. 2.6 abgebildet.

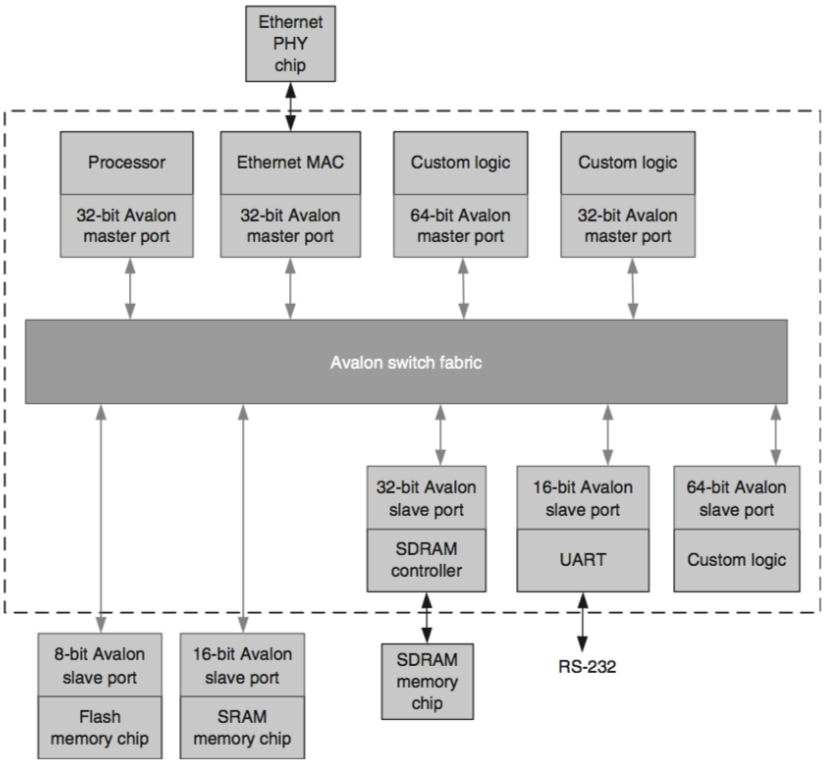


Abbildung 2.6.: Avalon-Bus basiertes System [6, S. 7–6]

Die Avalon-Spezifikation sieht sieben Schnittstellentypen vor [8, S. 4]:

- Avalon Streaming Interface (Avalon-ST) - unterstützt unidirektionalen Datenfluss (vgl. Abb. 2.2)
- Avalon Memory Mapped Interface (Avalon-MM) - unterstützt adressbasierte bidiriektionale Master-Slave Kommunikation

- Avalon Conduit Interface - fasst Signale, die nicht einem Avalon-Schnittstellentyp zuzuordnen sind, zusammen. Eine Verbindung im Platform Designer bzw. Export zur manuellen Verknüpfung ist möglich
- Avalon Tri-State Conduit Interface (Avalon-TC) - unterstützt Verbindungen zu Off-Chip Komponenten. Mehrere Peripheriegeräte können Pins durch Signalmultiplexing teilen
- Avalon Interrupt Interface - ermöglicht Komponenten Ereignisse an andere Komponenten zu vermitteln
- Avalon Clock Interface - Schnittstelle für Taktein- bzw. Ausgang
- Avalon Reset Interface - Schnittstelle für Reset-Logik

Avalon nutzt separate Adress-, Daten-, und Steuerleitungen. [6, S. 7–6] In Tabelle 2.1 sind die für diese Arbeit wichtigsten Avalon-MM Signale aufgelistet.

Signal	**Beschreibung**
address	Adresse
byteenable	Byte-Selektor
read	Signalisiert einen Lesezugriff
write	Signalisiert einen Schreibzugriff
readdata	Vom Slave übertragenes Datenwort
writedata	An den Slave übertragenes Datenwort
waitrequest	An den Master übermitteltes Wartesignal
response	Optionaler Antwortcode

Tabelle 2.1.: Avalon-MM Signale [8, S. 14–16]

2.3. Plasma CPU

Die Plasma CPU ist ein kleiner, synthetisierbarer 32-bit Reduced Instruction Set Computer (RISC) Mikroprozessor. Derzeit führt es einen Webserver mit einem Interruptcontroller, Universal Asynchronous Receiver-Transmitter (UART), Static Random Access Memory (SRAM) oder Double Data Rate Synchronous Dynamic Random Access Memory (DDR SDRAM) Controller, und einem Ethernetcontroller aus. Die

Plasma CPU führt bis auf unausgerichtete Lade und Speicheroperationen alle Microprocessor without Interlocked Pipeline Stages (MIPS) I User-Mode Instruktionen aus [1].

Der auf dem Plasma ausgeführte Webserver ist unter der angegebenen Adresse *https://plasmacpu.no-ip.org:8080/* [1], geprüft am 22. April 2018, nicht erreichbar.

In Abb. 2.7 ist das Block-Diagramm der Plasma-CPU abgebildet.

2.4. ZedBoard development kit

Das ZedBoard von Digilent ist ein Entwicklungs- und Evaluationsboard mit einem verbauten Xilinx Zynq-7000 SoC mit Peripherie. Das Zynq SoC ist in 3 Blöcke aufgeteilt, wie in Abb. 2.8 detailliert aufgeführt ist:

- Multiplexed I/O (MIO)
- Processing System (PS)
- Programmable Logic (PL)

Das PS verfügt über einen ARM-Cortex™A9 Prozessor und kann auf den DDR SDRAM zugreifen. Das PL stellt ein FPGA mit Peripherie dar und kann sowohl als domänenspezifischer Beschleuniger für das PS als auch als eine autarke Einheit konfiguriert werden. Das MIO fasst die sowohl von der PS als auch PL nutzbare Peripherie - exklusiver Zugriff vorausgesetzt - zusammen.

Aus dieser Struktur wird ersichtlich, dass ein Zugriff auf sämtliche Ressourcen aus der PL nicht ohne Weiteres möglich ist. [9, S. 16] Dazu gehören unter Anderem der DDR SDRAM, der SD-Kartenleser sowie der UART-Controller. Ein Zugriff auf diese ist in der einfachsten Variante nur über das Einrichten und Hochfahren des PS mit dem integrierten ARM-Prozessor und eines vollwertigen Betriebssystems möglich, was der Umsetzung einer eigenen CPU bzw. SoC nicht gerade dienlich ist. Einige Einschränkungen lassen sich durch den Einsatz der Peripheral Modules (PMODs) umgehen. Es wurden Module für Universal Serial Bus (USB)-UART erworben, sowie ein SD-Kartenleser-Modul. Wie in Abb. 2.8 dargestellt, sind die PMODs auch von der PL ansprechbar und somit nutzbar.

Auf dem ZedBoard sind unter Anderem auch ein Display-Modul, ein High-Definition Multimedia Interface (HDMI)-Transmitter sowie ein Video Graphics Array (VGA)-Ausgang verbaut. Diese sind ausschließlich von der PL aus ansprechbar.

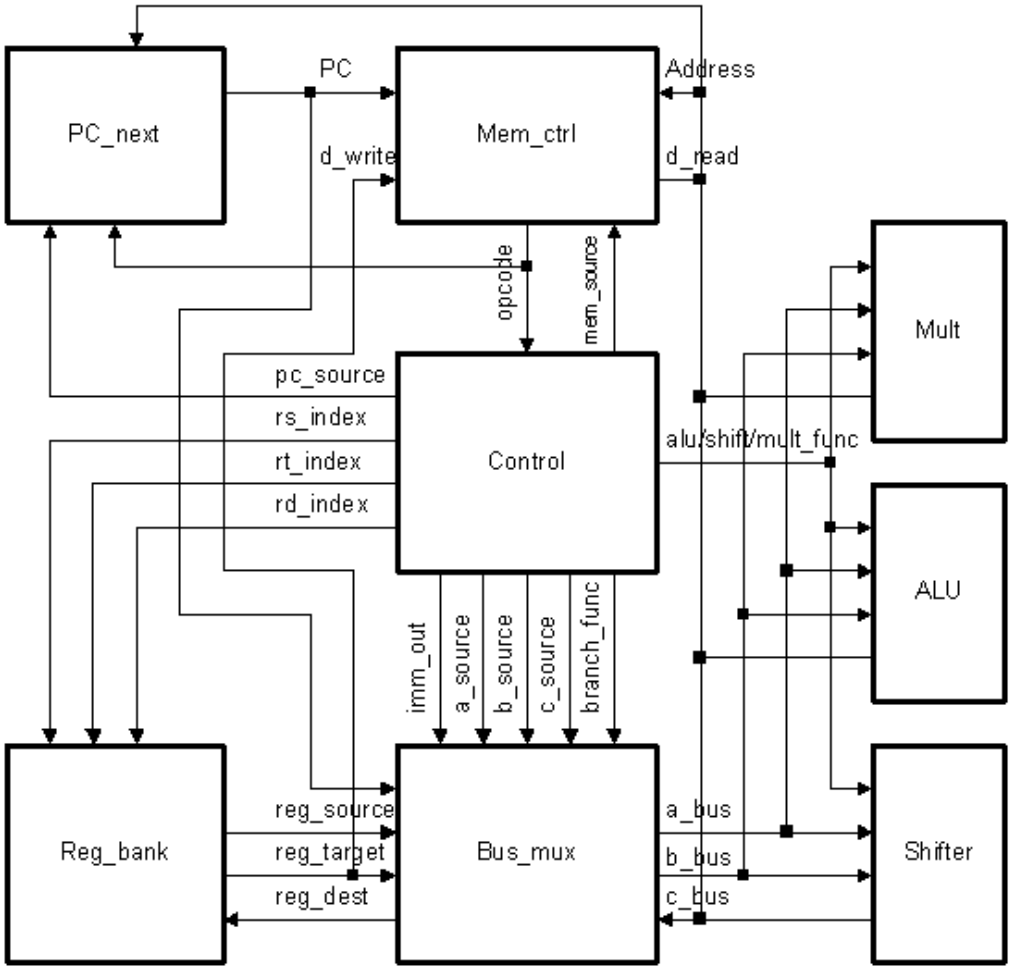


Abbildung 2.7.: Plasma-CPU Blockdiagramm [1]

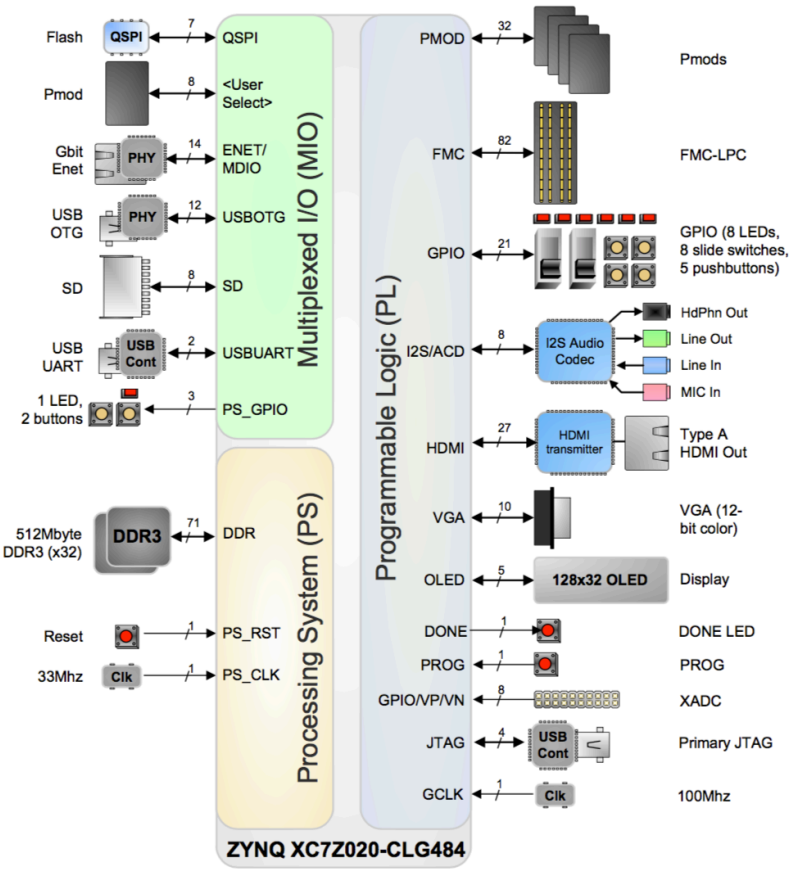


Abbildung 2.8.: ZedBoard Blockdiagramm [3, S. 3]

2.5. DE1-SoC development kit

Das DE1-SoC Board von Terasic ist ein Eintwicklungs- und Evaluationsboard mit einem verbauten Cyclone V FPGA-SoC.

Wenn man das in Abb. 2.9 abgebildete Blockdiagramm betrachtet und mit dem Blockdiagramm des ZedBoard in Abb. 2.8 vergleicht, so fallen starke Ähnlichkeiten auf. Diese sind darin begründet, dass bei beiden FPGAs ein ähnliches Konzebt verfolgt wird, bei dem sowohl ein ARM-Prozessor als auch ein FPGA mit Peripherie zur Verfügung gestellt wird.

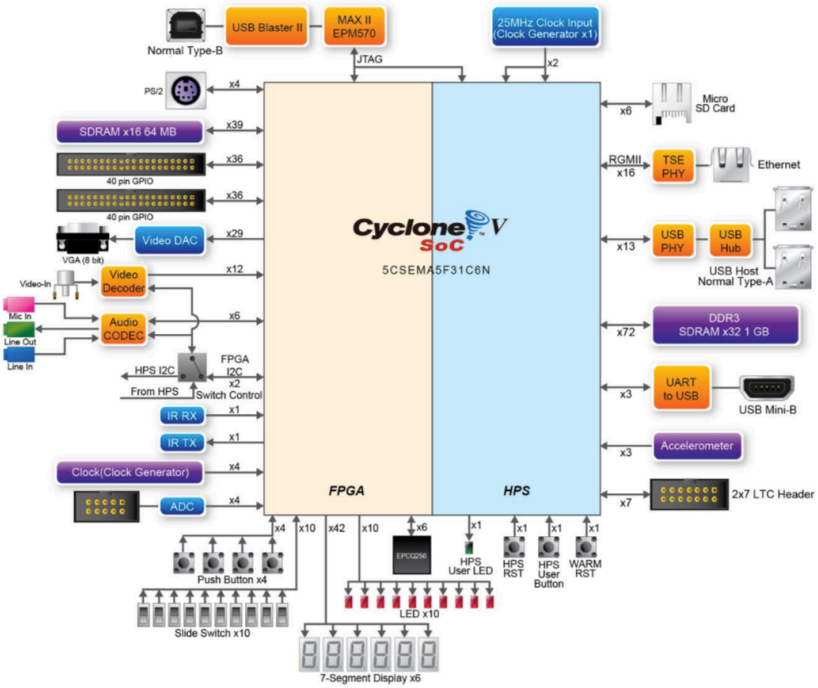


Abbildung 2.9.: DE1-SoC Blockdiagramm [4, S. 9]

Auf dem DE1-SoC-Board stehen folgende Komponenten zur Verfügung [4]:

- FPGA
  - Altera Cyclone®V SE FPGA
  - 64 MB SDRAM (16 bit Datenbus)
  - 4 Taster
  - 10 Schalter

  - 10 rote LEDs
  - 6 7-Segment-Anzeigen
  - 4 $50\,\mathrm{MHz}$ Oszillatoren
  - $24\,\mathrm{bit}$ Audio-CODEC mit Line- und Mikrofoneingängen und Line-Ausgang
  - VGA DAC mit VGA-Ausgang
  - TV Decoder (NTSC/PAL/SECAM) und TV-Eingang
  - PS/2 Maus-/Tastaturanschluss
  - Infrarot Sender/Empfänger
  - Zwei 40-Pin GPIO-Leisten
- Hard Processor System (HPS)
  - $800\,\mathrm{MHz}$ Dual-core ARM Cortex-A9 MPCore Prozessor
  - $1\,\mathrm{GB}$ DDR3 SDRAM ($32\,\mathrm{bit}$ Datenbus)
  - 1 Gigabit Ethernet PHY mit einem RJ45 Anschluss
  - 2-Port USB Host, normal Type-A USB Stecker
  - Micro SD-Kartenschacht
  - Beschleunigungssensor (I2C)
  - UART to USB, USB Mini-B Stecker
  - Warm-Reset und Cold-Reset-Taster
  - Ein benutzerdefinierte Taster, eine benutzerdefinierte LED
  - LTC 2x7 Erweiterungsstecker

In Abb. 2.10 ist die Oberseite des DE1-SoC-Boards mit Hervorhebung der einzelnen Komponenten abgebildet.

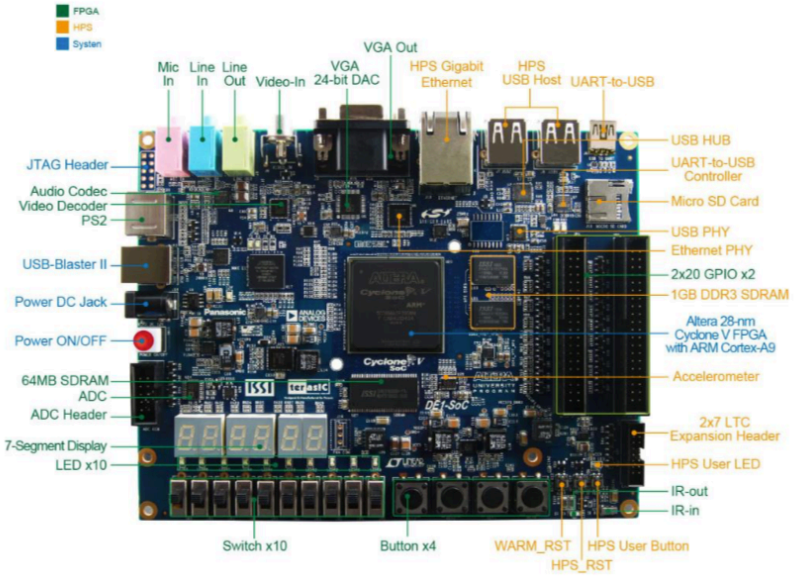


Abbildung 2.10.: DE1-SoC Oberseite [4, S. 7]

2.6. Entwicklungsworkflow mit Quartus

Um Hardware für einen FPGA entwickeln zu können, bedarf es einer passenden Entwicklungsumgebung. Da dem verwendeten Board ein Altera/Intel FPGA zugrundeliegt, wird die Entwicklungsumgebung „Quartus“ verwendet. Diese ist nach einer Registration kostenfrei in der Version „Quartus Prime Lite Edition 18.0“ im Downloadportal[2] von Intel verfügbar.

Die Umgebung beinhaltet mehrere Programme, die in dieser Arbeit verwendet werden.

Weiter beschreiben

2.7. Devicetree

2.8. Linux

[2] `http://fpgasoftware.intel.com/?edition=lite`

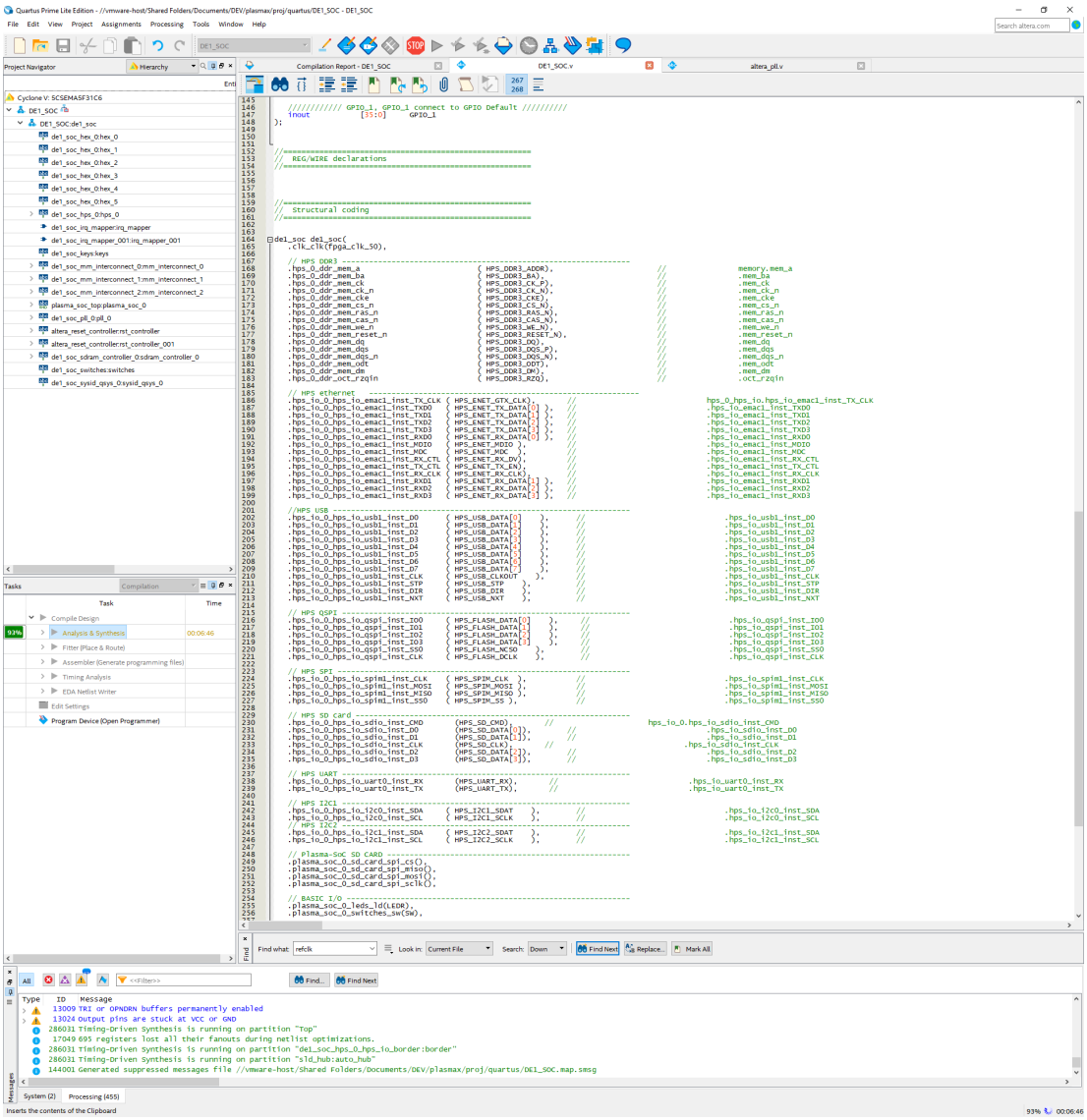


Abbildung 2.11.: Quartus Hauptprogramm

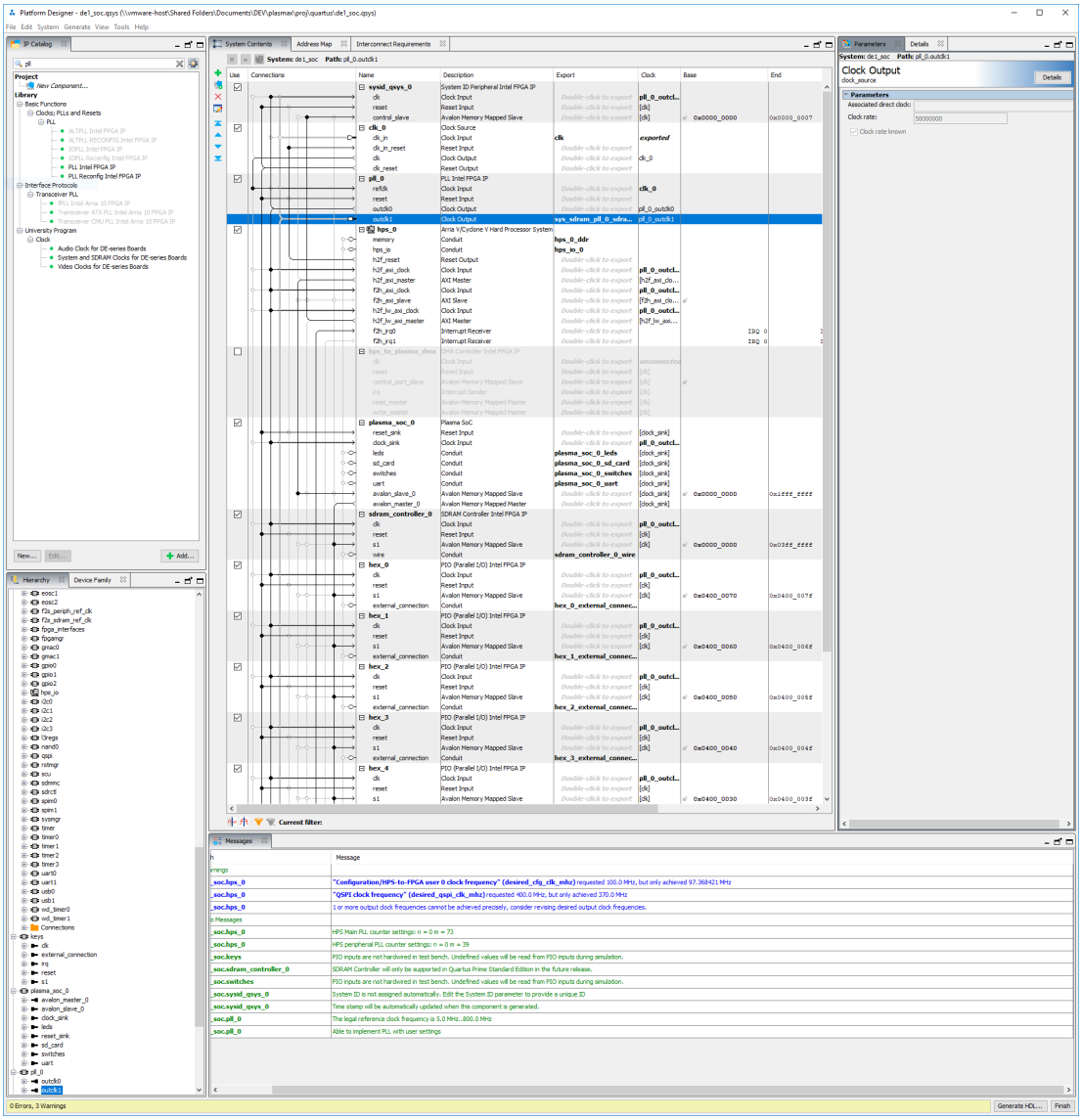


Abbildung 2.12.: Quartus Platform Designer

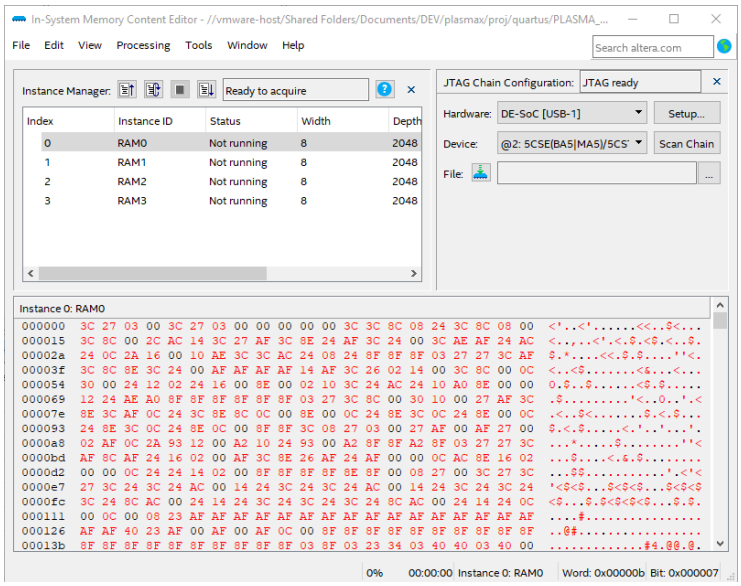


Abbildung 2.13.: Quartus In-System Memory Content Editor

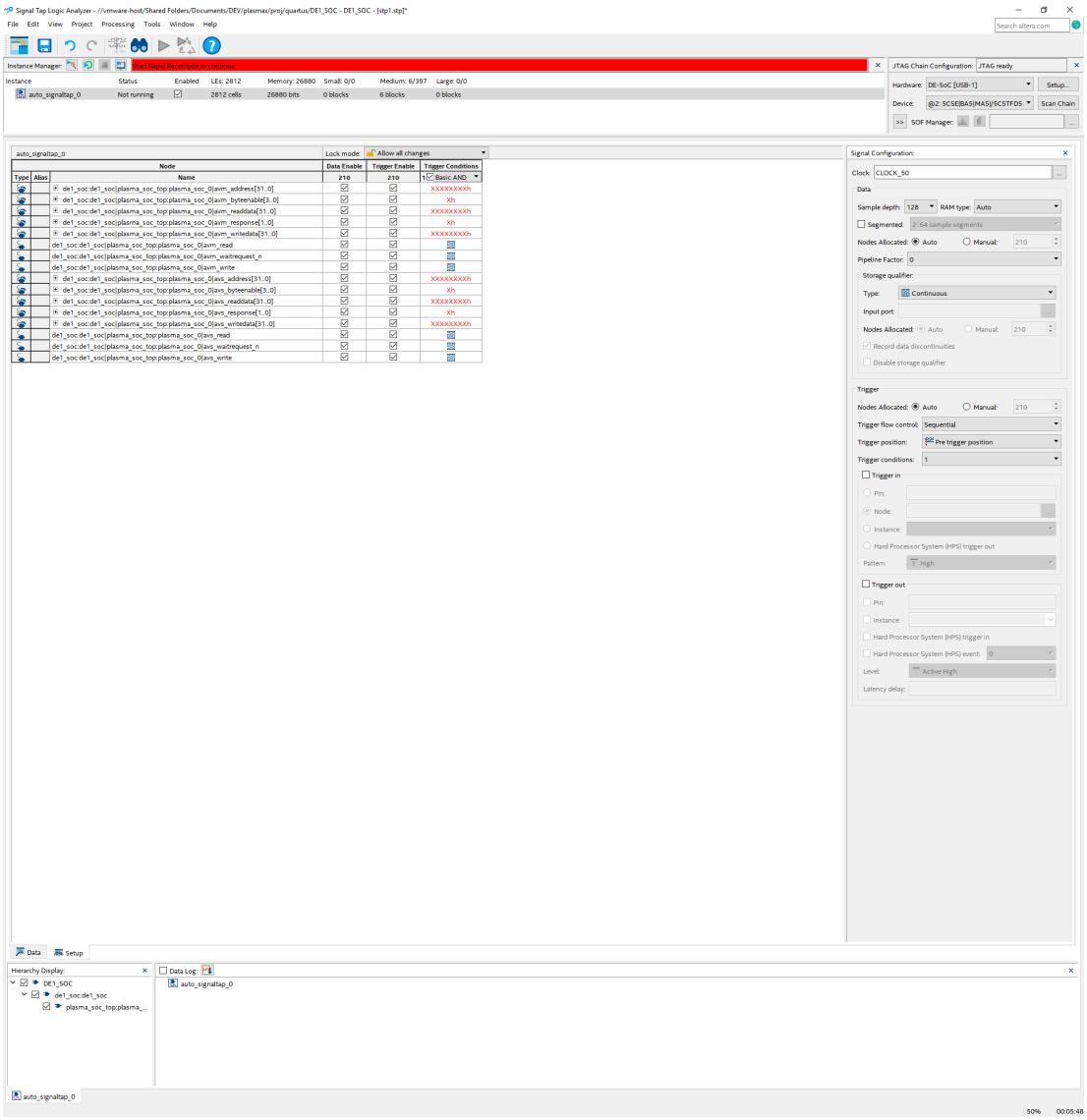


Abbildung 2.14.: Signal Tap Logic Analyzer

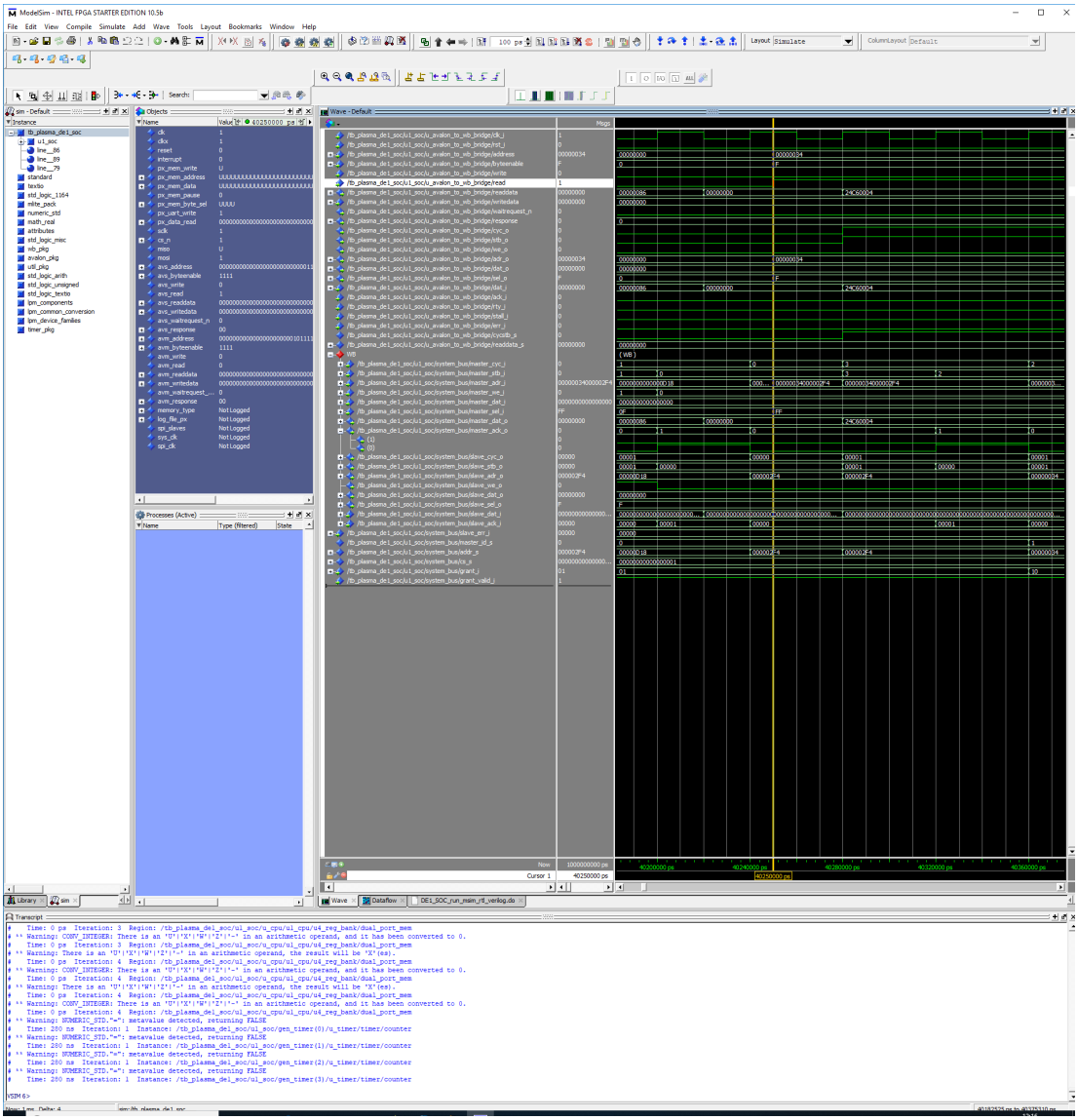


Abbildung 2.15.: Mentor Graphics Modelsim

3. Systemarchitektur

Dieses Kapitel beschreibt die vorhandene und vorgeschlagenen Architekturen und dient dem besseren Verständnis über die Umsetzung im Kapitel 4.

Im Abschnitt 3.1 wird die Architektur des Ausgangssystems in der Version 2.1 beschrieben.

Im Abschnitt 3.2 wird eine neue Architektur, die den Wishbone-Interconnect nutzt, neue Komponenten aufweist und speziell auf das Zedboard zugeschnitten ist, vorgestellt.

Im Abschnitt 3.3 werden zwei neue Architekturen vorgeschlagen. Die erste stellt einer Portierung auf das DE1-SoC Board dar, in der zweiten wird die erste Architektur erneut aufgegriffen und erweitert.

3.1. Plasma

Auf Abb. 3.1 ist die Architektur des Plasma-Systems im Ausgangszusatand (Version 2.1) abgebildet. Diese beinhaltet nur einen rudimentären Interrupt-Controller, zwei General-Purpose Input/Outputs (GPIOs), einen Counter sowie einen UART Controller. Außerdem ist der interne RAM als FPGA-interner Block-RAM implementiert und der externe RAM ungenutzt. Die Verknüpfung zwischen der Plasma-CPU und den Komponenten erfolgt über einen sehr einfachen Multiplexer mit Adressdekoder, dessen Erweiterung sich allerdings als äußerst umständlich erweist.

3.2. Plasma-SoC auf dem ZedBoard

Die erste Entwicklungsiteration erfolgt für das Digilent ZedBoard. Es werden ein Wishbone-Interconnect und die dazugehörigen Wrapper bzw. Adapter entwickelt. Außerdem werden neue Komponenten wie z.B. Zähler, ein Controller für das Display mit der Modellnummer UG-2832HSWEG04 sowie ein Serial Peripheral Interface

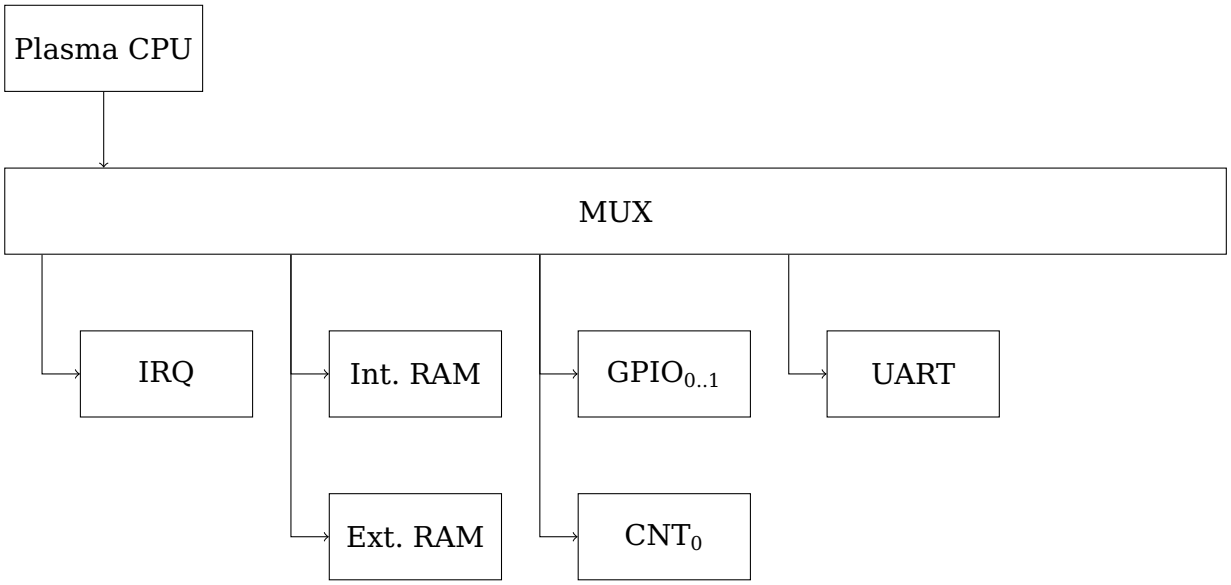


Abbildung 3.1.: Blockdiagramm des Plasma im Ausgangszustand

bus (SPI)-Controller eingeführt. Einen Überblick über die neue Systemarchitektur verschafft Abb. 3.2.

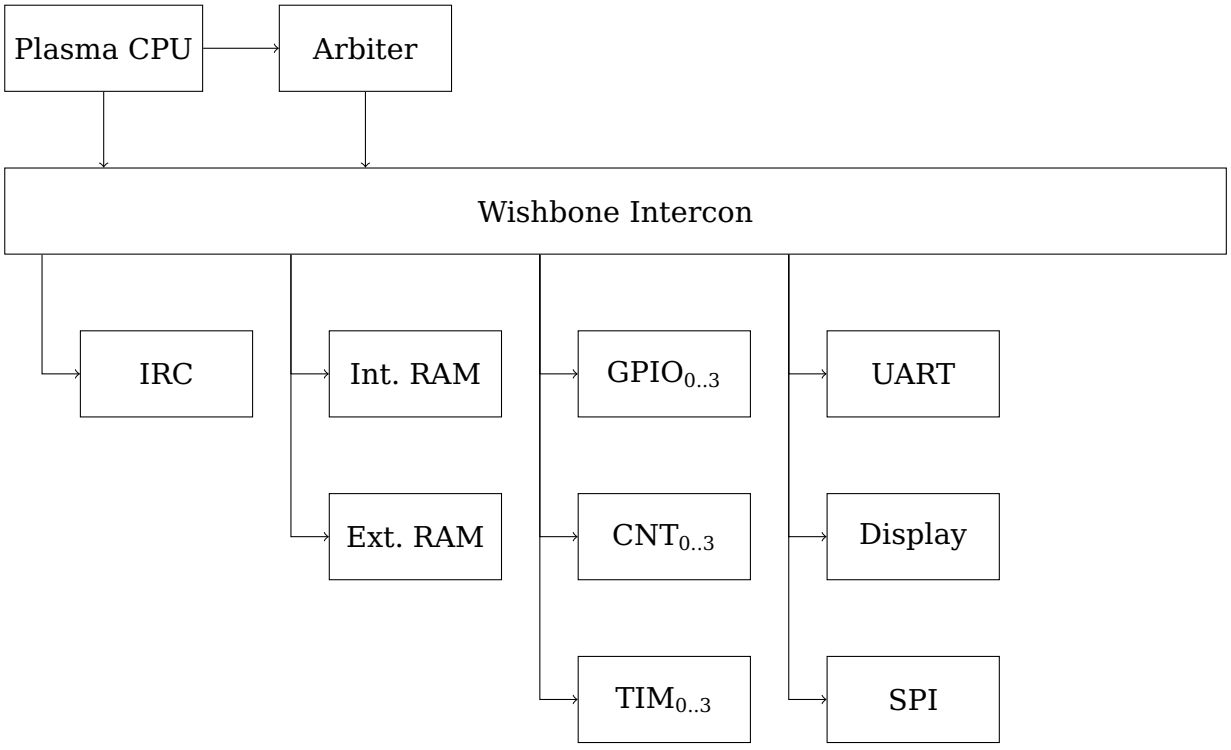


Abbildung 3.2.: Blockdiagramm des Plasma-SoC auf dem ZedBoard

3.3. Plasma auf dem DE1-SoC-Board

In der zweiten Iteration wird das vorhandene Plasma-SoC auf das neue Terasic DE1-SoC-Board portiert. Die dabei verfolgte Architektur ist auf Abb. 3.3 abgebildet. Da das Terasic DE1-SoC-Board nicht über ein Display sowie vom FPGA nutzbaren UART verfügt, werden diese Komponenten aus dem Wishbone-Bus und der Adresszuordnung entfernt. Ebenso wird auf die externe RAM-Schnittstelle verzichtet.

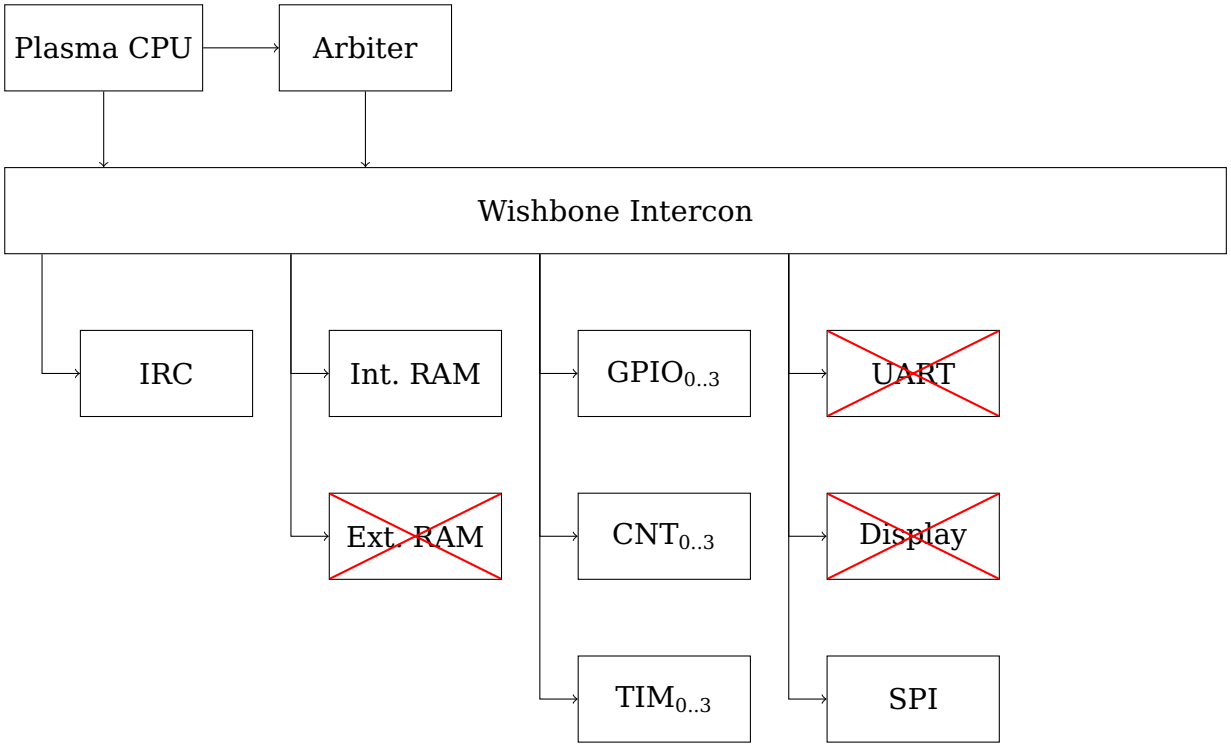


Abbildung 3.3.: Blockdiagramm des Plasma-SoC auf dem DE1-SoC

Die dritte Iteration sieht eine Erweiterung des auf das Terasic DE1-SoC-Board portierten Plasma-SoC vor. In diesem Schritt werden jeweils eine Avalon-Wishbone und eine Wishbone-Avalon Bridge als Master bzw. Slave zum Wishbone-Bus hinzugefügt, wie auf Abb. 3.4 dargestellt. Diese Neuerung und die Erstellung eines IP-Core ermöglicht eine Integration in das das im Platform Designer entworfene QSYS-System.

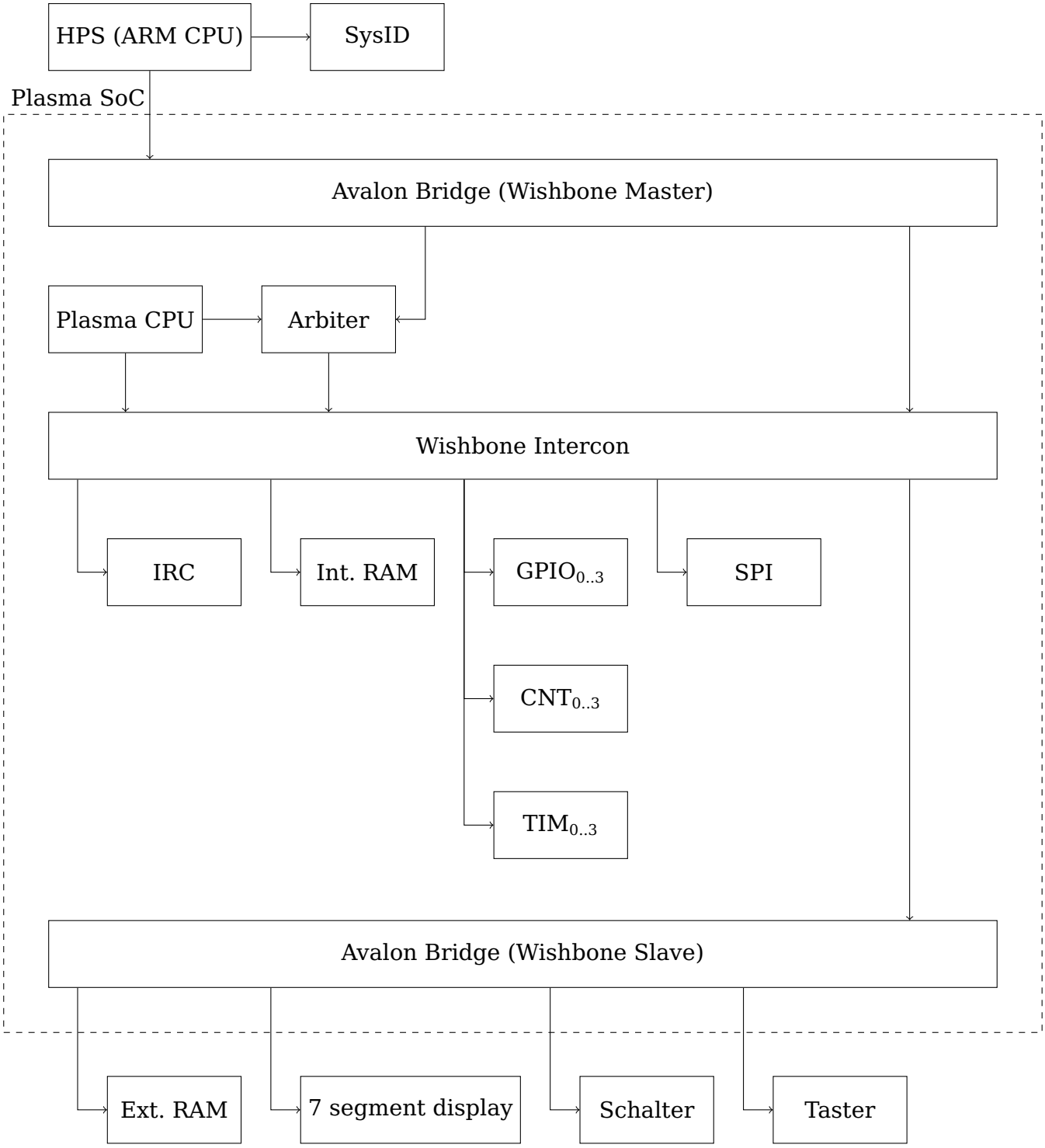


Abbildung 3.4.: Blockdiagramm des Plasma-SoC auf dem DE1-SoC mit Verbindung zu weiteren IP-Cores und ARM Prozessor über den Avalon-Bus

4. Umsetzung

In diesem Kapitel wird auf die Umsetzungsdetails der Hardware- und Softwareentwicklung, sowie der Systemeinrichtung eingegangen.

Im Abschnitt 4.1 werden wesentliche Etappen der Entwicklung des Wisbone-Interconnect, neuer Module und Wrapper für bereits vorhandene Komponenten geschildert, um nachvolgende Erweiterungen besser nachvollziehen zu können.

Die Entwicklung der Firmware für das Plasma-SoC wird in Abschnitt 4.2 umrissen.

Auf die Portierung des im Abschnitt 4.1 geschilderten SoC auf das in dieser Bachelorarbeit verwendete DE1-SoC Board wird in Abschnitt 4.3 eingegangen.

Im Abschnitt 4.4 wird das System aus Abschnitt 4.3 um zusätzliche Hardware-Komponenten erweitert, sodass der externe, $64\,\mathrm{MB}$ umfassende, RAM und auch die ARM-CPU verwendet werden können.

Im Abschnitt 4.5 wird die Erweiterung der Hilfssoftware geschildert.

Die Portierung der Plasma-Firmware und der Plasma-Beispielanwendung auf das DE1-SoC wird im Abschnitt 4.6 geschildert.

Der Abschnitt 4.7 beschreibt die Einrichtung eines Linux-Betriebssystems auf dem DE1-SoC Board und der dafür notwendigen Schritte.

Im Abschnitt 4.8 wird die Entwicklung der Linux Anwendungen, welche mit auf dem HPS ausgeführt werden und Zugriff auf das Plasma-SoC haben, beschrieben.

Im Abschnitt 4.9 wird die Entwicklung der Linux Treiber, welche einen alternativen Zugriff auf das Plasma-Soc erlauben, beschrieben.

4.1. Entwicklung des Wishbone-basierten Systems

Im Ausgangssystem stehen alle Komponenten durch manuelle Adressdekodierung und Datenmultiplex dem Plasma zur Verfügung. Jegliche Erweiterung erforderte

mehrere Eingriffe und Komponenten mit mehreren Kontroll- und Datenregistern waren nur sehr schwer einzubinden.

Es wird ein Shared-Bus (vgl. Abb. 2.4) gemäß der Wishbone-Spezifikation [7, S. 21] entwickelt. Als Entwicklungsgrundlage dient das „boschmitt/wishbone“[10] Projekt.

Sollte eine Adresse außerhalb des gültigen Bereiches angesprochen werden, so signalisiert der Bus einen Busfehler. Ein Fehler wird ebenfalls signalisiert, wenn eine korrekt ausgewählte Komponente innerhalb ihrer eigenen Adressdekodierung eine Zugriffsverletzung feststellt. Um einen Busfehler an den Kernel weiterzugeben, wird der Interrupt 31 verwendet (vgl. Tabelle 4.4).

Die zur partiellen Adressdekodierung nötigen Adressbereiche sind in Tabelle 4.1 aufgelistet.

Gerät	**Startadresse**	**Adressbereich**
Interner RAM	0x00000000	0x0FFFFFFF
Externer RAM	0x10000000	0x001FFFFF
IRC_0	0x20000000	0x1F
$UART_0$	0x20000100	0x0F
$Timer_{0:3}$	0x20000200	0x0F
	0x20000210	0x0F
	0x20000220	0x0F
	0x20000230	0x0F
$Zähler_{0:3}$	0x20000300	0x0F
	0x20000310	0x0F
	0x20000320	0x0F
	0x20000330	0x0F
$GPIO_{0:3}$	0x20000400	0x0F
	0x20000410	0x0F
	0x20000420	0x0F
	0x20000430	0x0F
SPI_0	0x20000500	0x0F
Display	0x40000000	0x7F

Tabelle 4.1.: Adresszuordnung

Um destruktive Änderungen an bereits vorhandenen Komponenten, wie der Plasma-CPU, RAM, UART, etc. zu vermeiden, werden Wishbone-Wrapper als zusätzliche Entitäten entwickelt. Dieses Entwurfsmuster wird auch für neue Module verwendet, da sie zum Teil auch als Untermodule in anderen Komponenten verwendet werden.

Zu den neu eingeführten Komponenten gehören der Interrupt-Controller, Zähler, Timer, GPIO-, UART-, SPI-Schnittstellen sowie der Display-Controller.

Es wird ein einheitliches Register-Schema für alle Komponenten entworfen (siehe Tabelle 4.2). An den unteren drei Adressen liegen nacheinander das Kontroll-, Status- und Datenregister. Die jeweils unteren beiden Bits sind für Control/Status Register (CSR) reserviert.

Offset	**Register**	**R/W**	**31 .. 2**	**1**	**0**
0x00	control	r/w	user defined	enable	reset
0x04	status	r	user defined	data available	ready
0x08	data	r/w	data		
⋮	⋮	⋮	⋮		

Tabelle 4.2.: Allgemeine Register-Map

Der Plasma Prozessor wird an das Wishbone-Interface angepasst. Dazu wird ein Wishbone-Master-Wrapper entwickelt. Dieser kontrolliert die *cyc* und *stb* Ausgangssignale, und reagiert auf *ack*, *stall*, *err*, *rty* Signale.

Der interne RAM wird an den Wishbone Bus angebunden. Der entsprechende Wrapper ist im Vergleich zu anderen Devices sehr einfach, da das in Tabelle 4.2 beschriebene Schema hier nicht befolgt werden muss und die Daten ohne weitere Adressdekodierung am Bus anliegen.

Der Interrupt Controller wird aus P. Tofahrns Abschlussprojekt [11] übernommen und angepasst. Für diesen wird ein Wishbone-Wrapper entwickelt. Zusätzlich waren Änderungen an der Plasma-CPU notwendig, da die CPU „bei einem Reset an Adresse 0 und bei einem Interrupt an 0x3C“ [11, S. 5] sprang.

Für sämtliche Komponenten werden Interrupts definiert. Da der neue Interrupt-Controller mit 32 Kanälen parametrisiert ist, kann man 32 verschiedene IRQs definieren. Diese sind in Tabelle 4.4 aufgelistet. Dabei sind die IRQs 30 bis 15 nicht zugeordnet und für künftige Erweiterungen gedacht.

Ein neuer Zähler mit neuen Features wird entwickelt. Bei diesem sind die Zählerrichtung, Reload-Wert sowie der Auto-Reload zur Laufzeit konfigurierbar. Bei Erreichen eines Grenzwertes (Aufwärts: reload - 1, Abwärts: 0) wird ein Interrupt ausgelöst. Die Register-Map ist in Tabelle 4.5 dargestellt.

Die Timer-Komponente baut auf dem obengenannten Zähler auf und erbt dessen wesentlichen Eigenschaften. Der Reload-Wert wird intern - abhängig von der konfigu-

Offset	Register	R/W	31 .. 2	1	0
0x00	control	r/w		enable	reset
0x04	status	r	unused		
0x08	data	r/w	data		
0x0C	imm flags	r	immediate interrupt flag state		
0x10	clear	r/w	clear interrupt flag		
0x14	invert	r/w	polarity		
0x18	mask	r/w	interrupt mask		
0x1C	edge	r/w	edge detection		

Tabelle 4.3.: Interrupt Controller Register-Map

IRQ	31	30:15	14	13:10	9:6	5:2	1	0
	Busfehler	$\emptyset$	Display	$GPIO_{3:0}$	$Zähler_{3:0}$	$Timer_{3:0}$	$UART_w$	$UART_r$

Tabelle 4.4.: IRQs

Offset	Register	R/W	31 .. 4	3	2	1	0
0x00	control	r/w		autoreload	direction	enable	reset
0x04	status	r				overflow	ready
0x08	data	r	current counter value				
0x0C	reload	r/w	reload value				

Tabelle 4.5.: Zähler Register-Map

rierten Zeiteinheit - modifiziert. Bei Erreichen des Schwellwertes wird analog zum Zähler ein Interrupt ausgelöst.

Offset	**Register**	**R/W**	**31 .. 6**	**5:3**	**2**	**1**	**0**
0x00	control	r/w		unit (1)	autoreload	enable	reset
0x04	status	r				overflow	ready
0x08	data	r	current timer value				
0x0C	reload	r/w	reload value				
Anmerkungen: 1. Erlaubte Zeiteinheiten sind aus Tabelle 4.7 zu entnehmen							

Tabelle 4.6.: Timer Register-Map

Einheit	**Code**	**Reload-Wert**
$1\,\mathrm{tick}$	0	1
$1\,\mathrm{ns}$	1	$50 * 10^{-3}$
$1\,\mu\mathrm{s}$	2	$50 * 10^{0}$
$1\,\mathrm{ms}$	3	$50 * 10^{3}$
$1\,\mathrm{s}$	4	$50 * 10^{6}$

Tabelle 4.7.: Timer Zeiteinheiten Kodierung und Reloadwert [ticks]

Für den bereits im Plasma-System vorhandenen UART-Controller wird ein Wishbone-Wrapper entwickelt. Die zugrundeliegende Komponente wird fest für eine Systemfrequenz von $50\,\mathrm{MHz}$ konfiguriert, sodass eine Kommunikation mit $57\,600\,\mathrm{Bd}$ erfolgen kann.

Offset	**Register**	**R/W**	**31 .. 2**	**1**	**0**
0x00	control	r/w		enable	reset
0x04	status	r		data available	ready
0x08	data	r/w	data		

Tabelle 4.8.: UART-Controller Register-Map

Um Peripherie über GPIO ansteuern zu können, wird ein rudimentärer GPIO-Controller entwickelt. Dieser unterstützt einfache Ausgaben; die Eingaben sind jedoch über eine konfigurierbare Maske maskiert, sodass nur bei bestimmten Pins ein Interrupt ausgelöst wird.

Offset	Register	R/W	31 .. 2	1	0
0x00	control	r/w		enable	reset
0x04	status	r		data available	ready
0x08	data	r/w	data		
0x0C	mask	r/w	mask		

Tabelle 4.9.: GPIO-Controller Register-Map

Basierend auf den SPI Controllern aus den „ZedBoard OLED Display“ [12] und „NexysVideo/GPIO“ [13] Projekten wird ein generischer SPI-Master entwickelt. Konfigurierbar sind die Anzahl Slaves, Datenwortbreite, System- und SPI-Frequenz. Aus der System- und SPI-Frequenz wird der Faktor für den Frequenzteiler gewonnen.

Offset	Register	R/W	31 .. 2	1	0
0x00	control	r/w		enable	reset
0x04	status	r		overflow	ready
0x08	data	r/w	data		
0x0C	address	r/w	SPI slave address		

Tabelle 4.10.: SPI Controller Register-Map

Aufbauend auf Michael Mattioli’s Projekt „ZedBoard OLED Display“ [12] wird der Display-Controller entwickelt. Der Display-Controller unterstützt derzeit nur den Textmodus, d.h der Inhalt des VRAM wird als ASCII-Zeichen interpretiert. Weitere Details sind der Registerzuordnung in Tabelle 4.11 zu entnehmen.

Offset	Register	R/W	31 .. 6	5	4	3	2	1	0
0x00	control	r/w		clear	flush	text	imm	enable	reset
0x04	status	r							ready
0x08:0x208	data	w	data						

Tabelle 4.11.: Display-Controller Register-Map

Bit	Name	Beschreibung
0	reset	Setzt das Display zurück
1	enable	Aktiviert das Display
2	imm	Datenregister wird als Display-Commandos interpretiert
3	text	VRAM wird als ASCII-Zeichen interpretiert
4	flush	Schreibt das VRAM an das Display
5	clear	Leert den VRAM

Tabelle 4.12.: Control Register Bits

4.2. Entwicklung der Plasma-Firmware

> Solltest du jemals ein Betriebssystem entwickeln, nenne es doch KebabOS - so wie unser KebabJS-Framework.
>
> Öner Caniperk

Zur Inbetriebnahme des Plasma-SoC benötigt man eine Firmware. Die neu entwickelten Geräte erfordern Treiber, um von der Software angesteuert werden zu können.

4.2.1. Treiber

Die entwickelten Treiber bauen auf memory-mapped Strukturen auf. Dies hat den Vorteil, dass man nicht jede einzelne Speicher- bzw. Geräteadresse über eine Basisadresse+Offset ansprechen muss. Es reicht, die Basisadresse der Struktur zu setzen und die Strukturfelder werden vom Compiler automatisch auf inkrementierte Adressen abgebildet. Es wird eine Übereinstimmung der verwendeten Datenwortbreite und Reihenfolge zur Definition in Tabelle 4.2 vorausgesetzt.

```
1 typedef struct
2 {
3 reg32_t control;
4 reg32_t status;
5 reg32_t data;
6 } volatile device_t;
```

Es wäre hilfreich, zur Laufzeit erkennen zu können, welches Gerät angesprochen wird. Dafür wurde ein Gerätedeskriptor entwickelt, welcher eine Gerätestruktur aufnehmen kann und mit einer Gerätetypkennung versieht. Die aktuell unterstützen Geräte sind:

- Zähler
- Timer
- Interrupt Controller
- GPIO Port
- UART Controller
- SPI Controller
- Display Controller
- Dateisystem
- Unbekanntes Gerät (Standard-Wert)

Zusätzlich zu den Strukturen, welche Geräte-Register abbilden, wurden allgemeine Funktionen entwickelt, die eine Arbeit mit allen kompatiblen Geräten vereinfachen. Die umgesetzten Funktionen sind in Tabelle 4.13 aufgelistet.

Die Funktionen des UART-Treibers werden durch Aufrufe der in Tabelle 4.14 aufgelisteten Funktionen umgesetzt.

Der Lese- bzw. Schreibzugriff ist auch über eine über eine UART-Datei möglich. Diese kann man über `FILE* uf = fopen("dev/uart0", /*modus*/);` öffnen.

Der Timer wird ebenfalls über einen Treiber konfiguriert. Ein Möglichkeit, die Zeiteinheit zu setzen (tick, ns, µs, ms, s) ist gegeben. Der Timer kann als selbstzurücksetzend konfiguriert werden, das heißt dass er kontinuierlich weiterzählt und bei jedem Erreichen des Schwellwertes einen Interrupt setzt und wieder beim Initialwert beginnt. Die Treiberfunktionen sind in Tabelle 4.15 aufgelistet.

Mithilfe des Zählertreibers wird die Konfiguration vereinfacht. Es werden Strukturen und Funktionen eingeführt, um einen neuen Schwellwert zu setzen, den Zähler zurückzusetzen und zu aktivieren. Im Gegensatz zu anderen Geräten, dürfen hier nicht die generischen `kd_enable` und `kd_reset` Funktionen zum Aktivieren bzw. Zurücksetzen des Zählers verwendet werden, da der Schwellwert abhängig von der

Kernfunktion	Beschreibung
kd_await_ready kd_await_data	Blockierendes Warten auf Bereitschaft oder Daten
kd_reset kd_enable kd_disable	Gerät zurücksetzen, aktivieren und deaktivieren
kdd_read kdd_write	Datenregister des Geräts auslesen und beschreiben
kdc_read kdc_test kdc_write kdc_set kdc_unset	Kontrollregister des Geräts gänzlich oder über Bitmaske auslesen und beschreiben
kds_read kds_write	Statusregister des Geräts gänzlich oder über Bitmaske auslesen

Tabelle 4.13.: Allgemeine Treiberfunktionen

Funktion	Beschreibung	Kernfunktion
uart_reset	UART zurücksetzen	kd_reset
uart_enable	UART aktivieren	kd_enable
uart_disable	UART deaktivieren	kd_disable
uart_read	UART-Datenregister auslesen	kdd_read
uart_write	UART-Datenregister beschreiben	kdd_write

Tabelle 4.14.: Treiberfunktionen des Display-Controllers

Funktion	Beschreibung
timer_set_reload	Neuen Zählwert setzen
timer_set_unit	Zeiteinheit setzen
timer_set_autoreset	Zähler automatisch zurücksetzen

Tabelle 4.15.: Treiberfunktionen des Timers

Zählrichtung zurückgesetzt wird und es zu undefiniertem Laufzeitverhalten kommen kann. Die Treiberfunktionen sind in Tabelle 4.16 aufgelistet.

Funktion	Beschreibung
`counter_set_reload`	Neuen Zählwert setzen
`counter_reset`	Zähler zurücksetzen
`counter_enable`	Zähler aktivieren

Tabelle 4.16.: Treiberfunktionen des Zählers

Für den Display-Controller werden in Tabelle 4.17 aufgelistete Funktionen umgesetzt. Der Zugriff auf den Videopuffer erfolgt über die Dateisystem-Abstraktionsschicht (empfohlen) über `FILE* df = fopen("dev/display0", /*modus*/);` oder alternativ direkt über die Display-Struktur bzw. generische Treiberfunktionen.

Funktion	Beschreibung	Kernfunktion
`display_reset`	Display zurücksetzen	`kd_reset`
`display_enable`	Display aktivieren	`kd_enable`
`display_disable`	Display deaktivieren	`kd_disable`
`display_set_textmode`	Textmodus setzen	
`display_set_graphicmode`	Rastermodus setzen	
`display_flush`	Videopuffer darstellen	
`display_clear`	Displayinhalt leeren	

Tabelle 4.17.: Treiberfunktionen des Display-Controllers

Die umgesetzten Treiberfunktionen des Interrupt-Controllers sind in Tabelle 4.18 aufgelistet. Die IRQs, die verwendet werden dürfen, entsprechen exakt den Angaben aus Tabelle 4.4.

Funktion	Beschreibung
`irc_is_set`	Prüfen, ob IRQ gesetzt
`irc_clear`	IRQ zurücksetzen
`irc_set_mask`	Maske für IRQ setzen
`irc_set_pol`	Polarität für IRQ setzen
`irc_set_edge`	Flankenerkennung für IRQ setzen

Tabelle 4.18.: Treiberfunktionen des Interrupt-Controllers

4.2.2. Dynamisches Speichermanagement

Das dynamische Speichermanagement ist notwendig, um zur Laufzeit neue Speicherblöcke allozieren und Programme flexibler gestalten zu können. Da für den Plasma kein vollwertiges Betriebssystem verfügbar ist und aus Speichermangel auf die Verwendung der C-Standardbibliothek verzichtet wird, ist eine eigene Implementierung der `malloc` unf `free` Funktionen notwendig. Bei einem vollwertigen Betriebssystem würden diese über einen Syscall den Kernel anweisen, einen freien Speicherblock zurückzugeben. Auch darauf wird hier verzichtet, stattdessen rufen diese unmittelbar die `kmalloc` und `kfree` Umsetzungen der Firmware auf. Die Umsetzung, die von EmbeddedArtistry [14] übernommen wird, baut auf einer verketteten Liste auf und gibt den jeweils ersten der angeforderten Größe entsprechenden Speicherblock zurück.

4.2.3. I/O

Es wird eine neue Abstraktionsschicht zur Kommunikation mit den Geräten eingeführt. Diese bietet, ähnlich wie bei UNIX-artigen Betriebssystemen, dateibasierten Zugriff auf Geräte. Beispielsweise schreiben `fprint(display_file, "TEST")` und `fprint(uart_file, "TEST")` jeweils die Zeichenfolge „TEST“ in den Video- und UART-Puffer. Um dies zu ermöglichen, wird eine `FILE`-Datenstruktur entworfen, welche einen Gerätedeskriptor sowie Ein- und Ausgabepuffer und die aktuelle Lese/Schreibposition abstrahiert. Über Funktionen, die mit den Datei-Funktionen aus der C-Standardbiblithek kompatibel sind, lassen sich Dateistrukturen öffnen, auslesen und beschreiben. Essentiell hierfür ist die in Abschnitt 4.2.2 beschriebene dynamische Speicherallokation, mit der zur Laufzeit neue Dateistrukturen und Puffer alloziert werden können.

4.3. Portierung der Hardware für DE1-SoC

Im ersten Schritt wird mit dem DE1-SoC SystemBuilder ein neues Projekt für Quartus erstellt. Das Tool generiert zusätzlich zur Projektdatei eine Verilog-Datei mit der Top-Level-Entity und die entsprechende Datei mit den Pin-Zuordnungen (QSF). Es werden, wie in Abb. 4.1 zu sehen ist, alle Optionen ausgewählt, damit die Pin-Zuordnung vollständig ist und bei künftigen Erweiterungen nicht angepasst werden muss.

In der Top-Level-Entity werden die Signalzuordnungen für VGA, Audio, Video, Infrarotsender, A/D Wandler und PS2 auskommentiert, da sie nicht gebraucht werden.

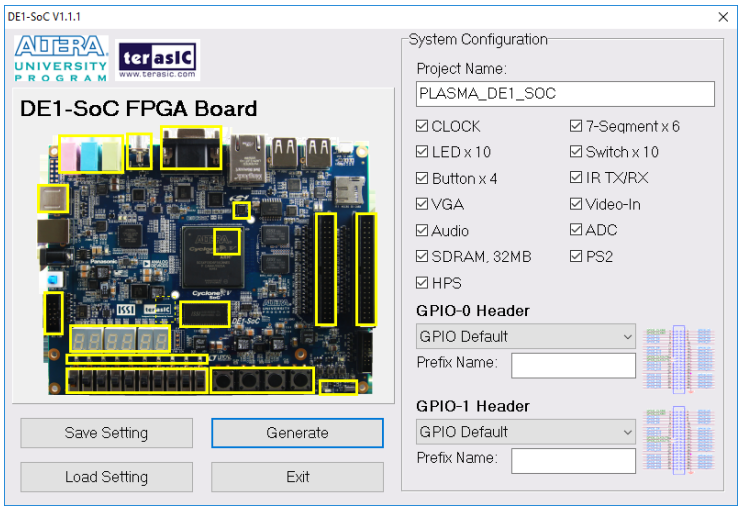


Abbildung 4.1.: DE1-SoC SystemBuilder

> Die aktuelle Revision des Entwicklungsboards hat bereits einen 64 MB SDRAM-Chip verbaut, die Angabe im SystemBuilder (32 MB SDRAM) wurde allem Anschein nach nicht aktualisiert.

Im zweiten Schritt wird das Quartus-Projekt überprüft und die tatsächliche Zielplattform zugewiesen, wie auf Abb. 4.2 dargestellt.

Darauffolgend wird der Platform Designer aufgerufen und über *File →New Component* ein neuer IP-Core erstellt.

Auf IP-Core erstellung eingehen

Leider kann ein mit dem Platform Designer erstelltes System nicht in Modelsim simuliert werden. Die Ursache dafür ist unklar. Darum wird ein neues Projekt „quartus_no_qsys“ angelegt, welches ausschließlich das Plasma-System ohne QSYS-Komponenten beinhaltet und Simulationszwecken dient.

4.3.1. RAM

Im bestehenden System werden Xilinx-Spezifische RAM-Blöcke verwendet, dabei sind mehrere RAM-Typen für Altera bzw. Xilinx konfigurierbar. Es gilt, eine passende Variante für den Altera-FPGAs zu finden.

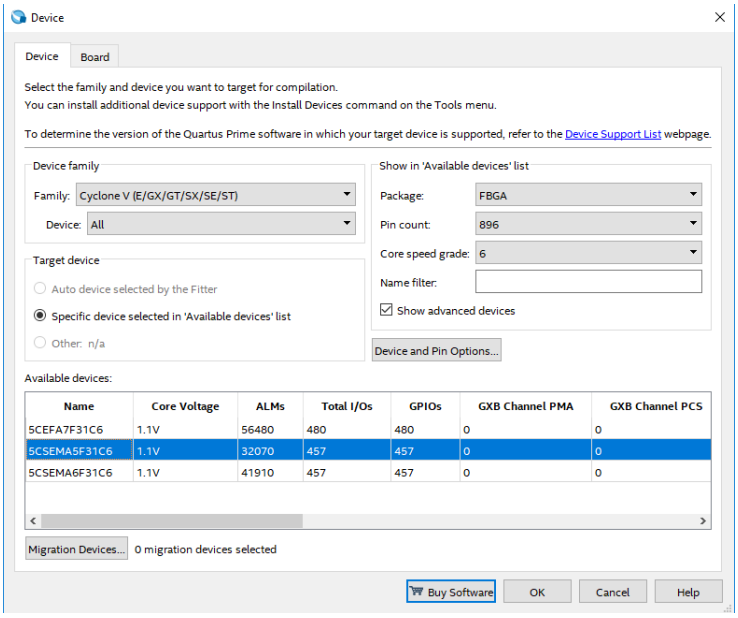


Abbildung 4.2.: Quartus Boardeinstellungen

Dabei wird zunächst versucht, durch Inferring, den RAM zu implementieren. Dieses Verfahren erweist sich jedoch als ungeeignet, da bei der Synthese der gesamte im Entwicklersystem verfügbare Arbeitsspeicher inkl. Swap aufgebraucht wird:

```
Error (114016): Out of memory in module quartus\_map.exe (24430
↪  megabytes used)
```

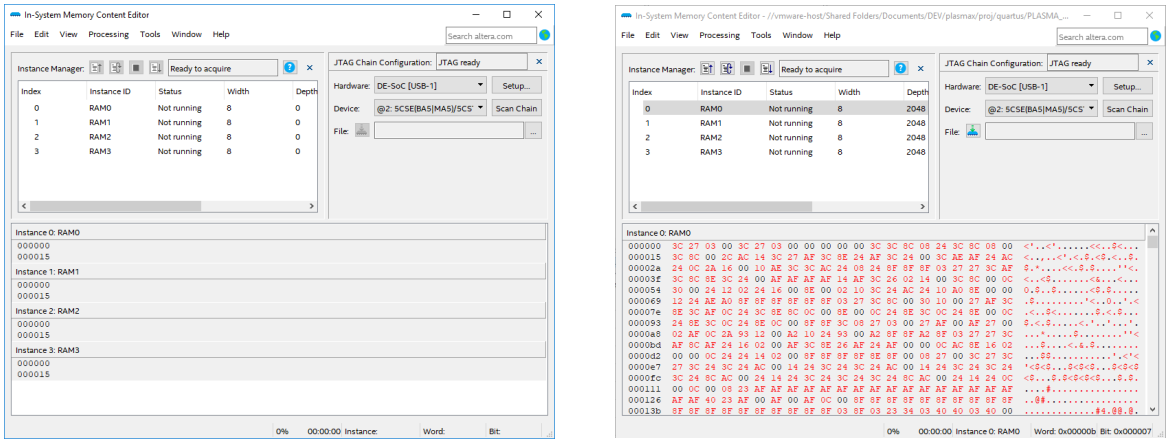
```
Error (114016): Out of memory in module quartus\_map.exe (44107
↪  megabytes used)
```

Als Nächstes werden die Altera Library of parameterized modules (LPM) Bausteine untersucht. Die Synthese verläuft zwar erfolgreich, jedoch wird das Beispielprogramm nicht ausgeführt. Quartus bietet mit dem In-System Memory Content Editor [15] eine Möglichkeit, den Speicherinhalt zu überprüfen und ggf. zu modifizieren. Dazu muss der LPM-RAM-Instanziierung ein zusätzliches Parameter hinzugefügt werden: *lpm_hint => „ENABLE_RUNTIME_MOD = YES, INSTANCE_NAME = <instantiation name>“* [15, S. 15–2]. Nach einem erneuten Testlauf stürzt der Memory Content Editor beim Versuch, den Speicherinhalt zu Laden, ab. Auffällig ist, dass die Blocktiefe 0 beträgt, wie in Abb. 4.3a zu sehen ist. Dieser Wert ist ungültig [16, S. 4–9] und muss korrigiert werden. Aus dem Synthese-Log ergibt sich der zu setzende Parameter *LPM_NUMWORDS*:

```
Info (12133): Instantiated megafunction (...) with the following
↪  parameter:
```

```
Info (12134): Parameter "LPM_WIDTH" = "8"
Info (12134): Parameter "LPM_WIDTHAD" = "11"
Info (12134): Parameter "LPM_NUMWORDS" = "0"
Info (12134): Parameter "LPM_INDATA" = "REGISTERED"
Info (12134): Parameter "LPM_ADDRESS_CONTROL" = "REGISTERED"
Info (12134): Parameter "LPM_OUTDATA" = "UNREGISTERED"
Info (12134): Parameter "LPM_FILE" =
 ↪   "../../build/sw/examples/sample_ram_image.vhd_block_1.mif"
Info (12134): Parameter "LPM_TYPE" = "LPM_RAM_DQ"
Info (12134): Parameter "USE_EAB" = "ON"
Info (12134): Parameter "INTENDED_DEVICE_FAMILY" = "UNUSED"
Info (12134): Parameter "LPM_HINT" = "ENABLE_RUNTIME_MOD = YES,
 ↪   INSTANCE_NAME = RAM1"
```

Nach der Parametrisierung des Speichers ist der Inhalt vorhanden und auslesbar, wie in Abb. 4.3b dargestellt.

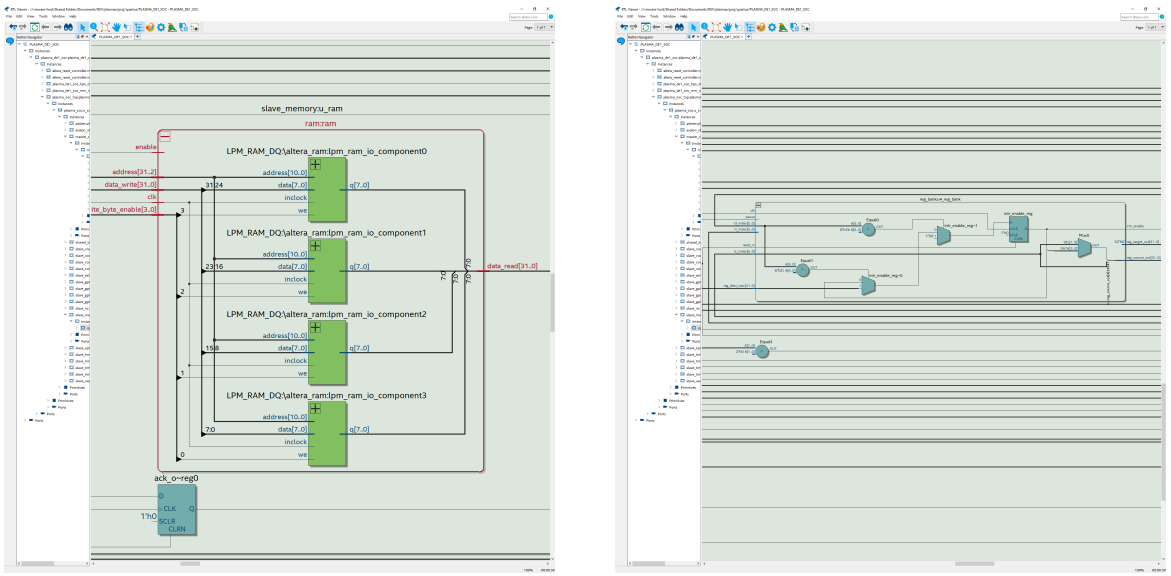


(a) Fehlerhafte LPM-Parametrisierung **(b)** Korrekte LPM-Parametrisierung

Abbildung 4.3.: Ausgelesener Speicherinhalt

4.3.2. Register-File

Das Beispielprogramm wird dennoch nicht ausgeführt. Die Vermutung, dass das als RAM-Block implementierte Register-File nicht synthetisiert wird, bestätigt sich durch eine Überprüfung der generierten Netzliste im RTL-Viewer. In Abb. 4.4 sind die Netzlisten des Arbeitsspeichers und der Register-Datei gegenübergestellt. Während in Abb. 4.4a die LPM-RAM-Elemente deutlich zu sehen sind, ist im Register-File auf Abb. 4.4b nur Steuerlogik ohne RAM-Blöcke erkennbar.



(a) RAM **(b)** REG_BANK

Abbildung 4.4.: Vergleich der RTL-Netzlisten

Es stellt sich heraus, dass für die Register-Datei der Speichertyp-Parameter mit dem ungeeigneten Wert „XILINX_16X“ konfiguriert ist und nicht synthetisiert wird. Altera LPM-RAM-Blöcke erweisen sich ebenfalls als ungeeignet, da sie mit dem verwendeten Cyclone V FPGA inkompatibel sind:

```
Warning (272007): DUAL PORT RAM for Cyclone V not possible with the
 ↪  altsyncram megafunction. Implementing using benchmarking mode,
 ↪  trying to tie up the inputs using available clocks. Output
 ↪  behavior will not be the same

Error (287078): Assertion error: Can't convert dual-port RAM for
 ↪  Cyclone V device family using altsyncram megafunction because
 ↪  Cyclone V supports only synchronous dual-port RAM
```

Es wird ein durch Inferring synthetisierbarer Tri-Port Speicher mit zwei Lese- und einem Schreibport für die Register-Datei ausgewählt.

4.3.3. Laufzeitverhalten

Durch Prüfung des Speicherinhalts während der Simulation fällt eine unsaktionierte Änderung des Arbeitsspeichers auf. Eine weitreichende Analyse zeigt, dass die Umsetzung des Arbeitsspeichers das vom Wishbone-Wrapper aus dem *STB*-Signal generierte *enable*-Signal nicht nutzt. Dies hat zur Folge, dass bei einem Schreibzugriff auf einen beliebigen Wishbone-Slave der auf dem Datenbus anliegender Wert in den Speicher übernommen wird. Nach der Behebung dieses Fehlers wird das Beispielprogramm korrekt ausgeführt und die im nächsten Abschnitt beschriebene Erweiterung der Hardware beginnt.

4.4. Erweiterung der Hardware für DE1-SoC

Remove UART from system? Adapt architecture?

4.4.1. Avalon-Wishbone Bridge

Wie bereits im Abschnitt 2.2.3 erwähnt, nutzt ein vom Platfom Designer generiertes SoC den proprietären Avalon-Interconnect. Das Plasma-SoC hingegen setzt den Wishbone-Interconnect ein. Damit beide Systeme sinnvoll verknüpft werden können, bedarf es einer Wishbone↔Avalon-Bridge.

Gerät	**Startadresse**	**Adressbereich**
Interner RAM	0x00000000	00001FFF
IRC_0	0x00005000	0x1F
$UART_0$	0x00005100	0x0F
$Timer_{0:3}$	0x00005200	0x0F
	0x00005210	0x0F
	0x00005220	0x0F
	0x00005230	0x0F
$Zähler_{0:3}$	0x00005300	0x0F
	0x00005310	0x0F
	0x00005320	0x0F
	0x00005330	0x0F
$GPIO_{0:3}$	0x00005400	0x0F
	0x00005410	0x0F
	0x00005420	0x0F
	0x00005430	0x0F
SPI_0	0x00005500	0x0F
Wishbone →Avalon Bridge	0x10000000	0x0FFFFFFF

Tabelle 4.19.: Plasma-SoC Adresszuordnung

In Tabelle 4.20 ist eine Gegenüberstellung der wichtigsten Avalon-MM und Wishbone Schnittstellensignale dargestellt. Erkennbar ist, dass Takt-, Adress- und Datensignale, sowie die Steuersignale zum Großteil 1:1 zugeordnet werden können [17]. Das *chipselect*-Signal gilt als veraltet [8, Appendix A]. Darum werden die Zuordnungen zu den Steuersignalen wie *cyc_i*, *stb_i* und *we_i* durch kombinatorische Logik und Zusatandsautomata hergeleitet. Die Timingdiagramme sind jeweils für die Avalon→Wishbone- und Wishbone→Avalon-Bridge in Abb. 4.5 und Abb. 4.6 abgebildet.

Wishbone Bus	**Avalon-MM**
clk_i	clk
rst_i	reset_n
cyc_i	read *or* write
stb_i	chipselect bzw. read *or* write
we_i	write *and not* read
sel_i	byteenable
adr_i	address
dat_i	readdata
dat_o	writedata
ack_o	waitrequest_n

Tabelle 4.20.: Avalon-MM und Wishbone Schnittstellen

4.4.2. Arbiter

Der bestehende Arbiter ist nicht funktionsfähig. Die Problematik fällt auf dem ZedBoard nicht auf, da der Wishbone-Bus dort mit nur einem Master betrieben wird. Der fehlerhafte Arbiter wird durch einen modifizierten, prioritätsbasierten Round-Robin Arbiter von Benjamin Krill [18] ersetzt.

Zunächst wird das Zweier-Komplement aus dem Request-Eingangssignal gebildet. Das Ergebnis und das Request-Eingangssignal werden mit dem UND-Operator bitweise verknüpft, wie in Gleichung (4.1) zu sehen ist. Das Endergebnis hat dann ein least significant bit (LSB) mit der höchsten Priorität [18].

$$request_{lsb} = request_i \wedge (\overline{request_i} + 1) \tag{4.1}$$

In Gleichung (4.2) wird aus dem Request-Eingangssignal und dem gespeicherten vorherigem Grant-Signal eine Maske gebildet, welche dann nach Anwendung der

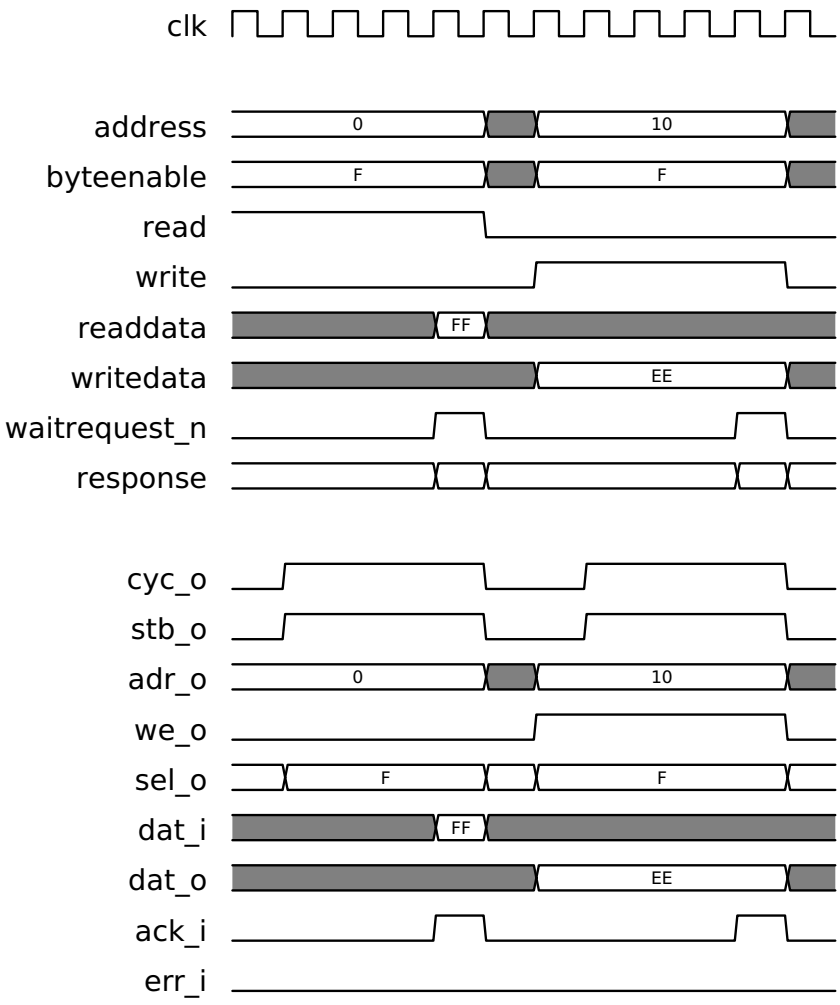


Abbildung 4.5.: Avalon→Wishbone-Bridge Timings

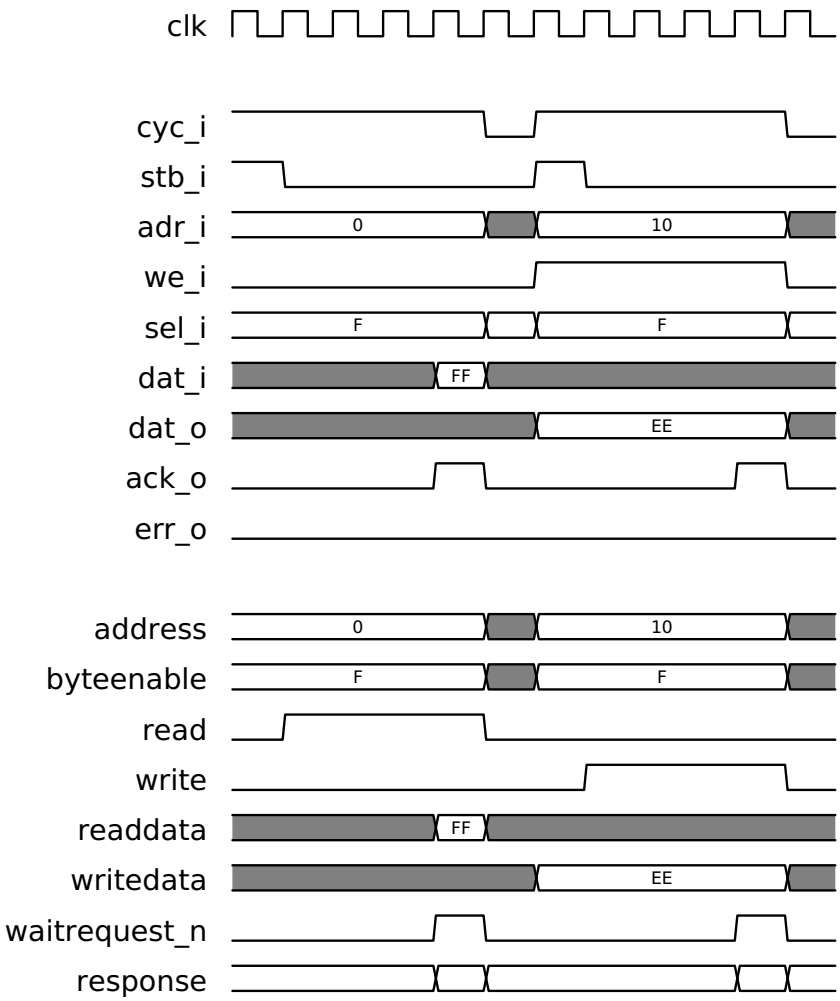


Abbildung 4.6.: Wishbone→Avalon-Bridge Timings

Gleichung (4.3) das neue Grant-Signal ergibt [18].

$$mask = request_i \wedge (\overline{grant_{prev} - 1) \vee grant_{prev}}) \tag{4.2}$$

$$grant_{next} = mask \wedge (\overline{mask} + 1) \tag{4.3}$$

Anschließend wird in Gleichung (4.4) entweder das neu generierte Grant-Signal bei einer gültigen Maske oder das Request-Signal mit der höchsten Priorität ausgewählt [18].

$$grant = \begin{cases} grant_{next}, & \text{if } mask \neq 0 \\ request_{lsb}, & \text{otherwise} \end{cases} \tag{4.4}$$

Zur Anbindung dieses Arbiters an den Wishbone-Bus wird dieser zusätzlich mit einem One-Hot dekodierten Grant-Signal versehen, da sich so der entsprechende Master mit den Daten-, Adress- und Steuersignalen einfacher selektieren lässt. Der Wishbone-Wrapper für die CPU wird ebenfalls angepasst, sodass die Wishbone-Steuersignale *CYC* und *STB* nur bei einer gültigen Buszuteilung generiert werden.

4.4.3. Wishbone-Bus

Nach der Einführung des verbesserten Arbiters fällt auf, dass die Adresse nach Auswahl des Masters erst einen Taktzyklus später dekodiert anliegt, was zum falschen Betriebsablauf führt. Als Konsequenz wird auf eine mit dem Takt verknüpfte Auswertung und Speicherung im Register verzichtet.

Beim simulationsbasierten Testen der Avalon→Wishbone-Bridge, welche als Wishbone-Master fungiert, fällt auf, dass ein vom Slave bereitgestelltes Datenwort erst zwei Taktzyklen nach dem *ACK*-Signal anliegt. Dadurch werden fehlerhafte Werte übernommen. Um dieses Problem zu umgehen, wird aus dem Wishbone-Bus die letzte an den Takt gebundene Komponente vom Takt enkoppelt. Dadurch verursacht der Bus keine nennenswerte Verzögerung mehr und die Daten liegen zeitgleich an den Ein- und Ausgängen an. Bei höheren Taktfrequenzen könnte es jedoch zu Problemen führen.

Die Plasma-CPU erwartet die Daten jedoch immer noch mit einem Taktzyklus Versatz. Darum wird im Wishbone-Wrapper der Plasma-CPU ein Register für die Datenleitung hinzugefügt, sodass das aktuell anliegende Datenwort erst einen Takt später zur Verfügung steht.

4.4.4. Externer SDRAM

Auf dem DE1-SoC-Board ist ein SDRAM-Chip mit der Modellnummer `IS42S16320D` von ISSI Technologies verbaut. Dieser fasst $64\,\mathrm{MB}$ (32Mx16) und ist mit dem FPGA über eine $16\,\mathrm{bit}$ breite Datenleitung sowie Kontroll- und Adressleitungen verbunden [4, S. 38]. Die Grundparameter lassen sich aus Tabelle 4.21 und Tabelle 4.22 entnehmen.

Parameter	**-5**	**-6**	**-7**	**Einheit**
Taktzykluszeit				
$\overline{\mathrm{CAS}}$ - Latenz = 3	5	6	7	ns
$\overline{\mathrm{CAS}}$ - Latenz = 2	10	10	7.5	ns
Taktfrequenz				
$\overline{\mathrm{CAS}}$ - Latenz = 3	200	167	143	MHz
$\overline{\mathrm{CAS}}$ - Latenz = 2	100	100	133	MHz
Zugriffszeit				
$\overline{\mathrm{CAS}}$ - Latenz = 3	5.0	5.4	5.4	ns
$\overline{\mathrm{CAS}}$ - Latenz = 2	6	6	5.4	ns

Tabelle 4.21.: Timing Parameter [19, S. 1]

Parameter	**16M x 32**	**32M x 16**	**64M x 8**
Konfiguration	4M x 32 x 4 banks	8M x 16 x 4 banks	16M x 8 x 4 banks
Bank Adress Pins	BA0, BA1	BA0, BA1	BA0, BA1
Autoprecharge Pins	A10/AP	A10/AP	A10/AP
Zeilenadresse	512(A0 – A8)	1K(A0 – A9)	2K(A0 – A9, A11)

Tabelle 4.22.: Adresstabelle [19, S. 1]

Um den Speicher in Betrieb zu nehmen, ist ein SDRAM-Controller nötig. Dieser steht als IP-Core im Platform Designer zur Verfügung. Folgende Parameter werden mit neuen Werten überschrieben [20, S. 6]:

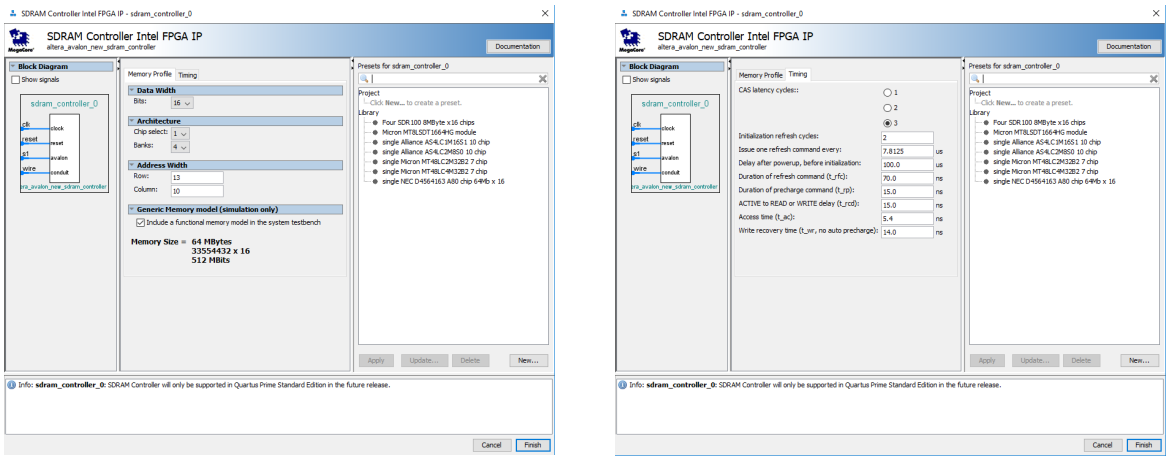
- Bits: 16
- Chip select: 1
- Banks: 4 (vgl. Konfiguration in Tabelle 4.22)
- Row: 13

- Column: 10 (vgl. Zeilenadresse in Tabelle 4.22)

Die Timings werden ebenfalls angepasst [20, S. 6]:

- CAS latency cycles: 3
- Issue one refresh command every: $7.8125\,\mu s$
- Duration of refresh command(t_rp): $15.0\,ns$
- Duration of precharge command(t_rcd): $15.0\,ns$
- Access time(t_ac): $5.4\,ns$ (vgl. Zugriffszeit in Tabelle 4.21)

In Abb. 4.7a und Abb. 4.7b sind die Parameter im Konfigurationsdialog des SDRAM-Controller-IP-Core abgebildet.



(a) Allgemeine Parameter **(b)** Timings

Abbildung 4.7.: Parametrisierung des SDRAM-Controllers

Die korrekte Funktion des SDRAM hängt auch von den physikalischen Charakteristika des Boards ab. Unter Anderem hat die Länge der Verbindungsleitungen zwischen dem SDRAM-Chip und dem FPGA einen Einfluss auf möglichen Taktversatz. Darum wird eine phase locked loop (PLL) benötigt, die den SDRAM-Takt dem Systemtakt unter Berücksichtigung der physikalischen Parameter um $3\,ns$ vorauseilen lässt [20, S. 13]. Dies kann durch Hinzufügen und geeignete Parametrisierung eines PLL-IP-Cores oder alternativ durch Hinzufügen des *Clocks Signals for DE-Series Board Peripherals*-IP-Cores geschehen. Die erste Variante setzt eine eine Berechnung der notwendigen Parameter oder eine aufwändige Analyse mit einem Oszilloskop voraus, sodass die Daten in einem korrekten Zeitfenster anliegen [21, 355ff]. Es wird die zweite Variante gewählt, da sie wesentlich einfacher ist und nur eine Auswahl der

Zielhardware im in Abb. 4.8 dargestellten Konfigurationsdialog erfordert. Es werden der Referenztakt von $50\,\mathrm{MHz}$, der gewünschete Systemtakt von $50\,\mathrm{MHz}$ sowie die Zielhardware (das DE1-SoC-Board) eingestellt.

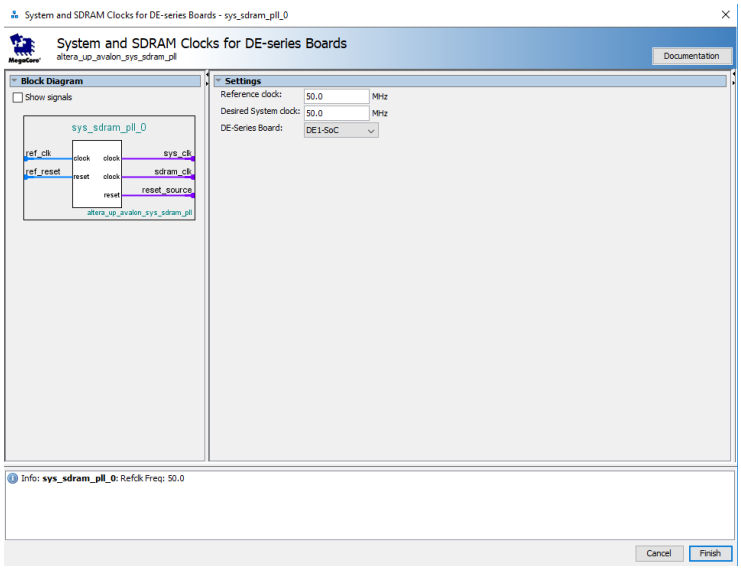


Abbildung 4.8.: Parametrisierung des *Clocks Signals for DE-Series Board Peripherals*-IP-Core

Ein wichtiger Aspekt ist, dass der verbaute SDRAM einen $16\,\mathrm{bit}$ breiten Datenbus hat. In Abb. 4.9 ist ein mit SignalTap aufgezeichneter Zugriff auf den externen SDRAM dargestellt. Von Interesse sind die blau umrahmten Wellenformen für *.../avm_read* und *.../avm_waitrequest_n*. Am oberen Rand der Abb. 4.9 ist die Zeit- bzw. Taktleiste zu sehen. Durch Abgleich der Wellenform mit der Zeitleiste lässt sich schlussfolgern, dass der gesamte Lesezyklus 6 Taktzyklen dauert. Da der SDRAM-Controller mit CAS=3 konfiguriert ist, liegen die Daten nach 3 Taktzyklen am Datenbus an. Entsprechend wird ein $32\,\mathrm{bit}$ Datenwort aus zwei $16\,\mathrm{bit}$ Transfers zusammengesetzt, welche dann zusammen 2x3 Taktzyklen ergeben.

4.5. Erweiterung der Hilfssoftware

Ein längere Zeit bestehendes Problem im in Abschnitt 3.2 beschriebenen System ist das scheinbar von der Programmgröße abhängige Laufzeitverhalten. Dies wird mit einem neuen Linker-Script behoben.

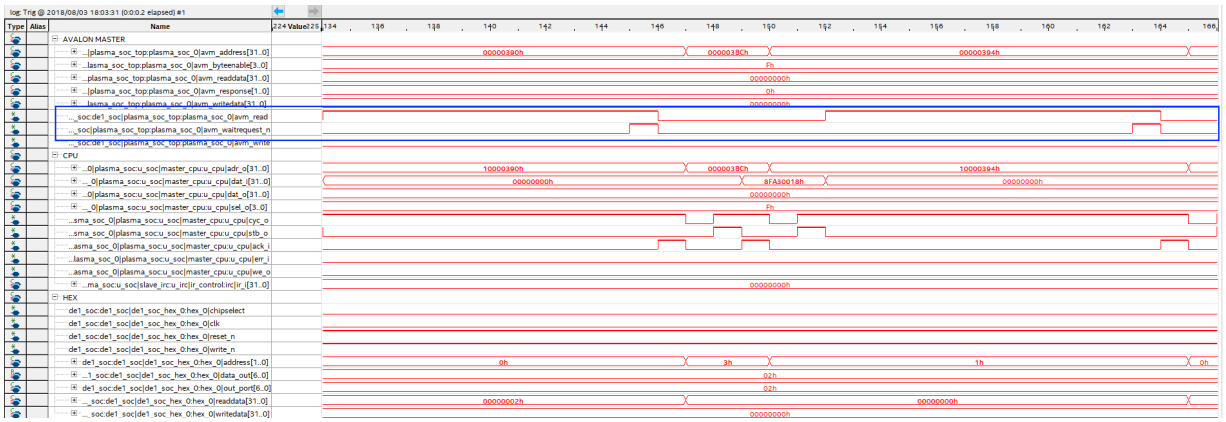


Abbildung 4.9.: Mit SignalTap aufgezeichneter Zugriff auf den externen SDRAM

4.6. Portierung der Plasma-Firmware und der Beispielanwendung

Die Portierung der Firmware erfolgt in Form einer neuen Systemdefinitionsdatei mit einer neuen Adresszuordnung und anderen Komponenten. Die bestehende Definition wird in die Datei `plasma_soc.zedboard.h` ausgelagert, und die neue in der Datei `plasma_soc.de1_soc.h` erstellt. Die CMake Build-Skripte werden um neue Einträge erweitert, welche die zwei verschiedenen Systemkonfigurationen repräsentieren. Dabei wird eine Präprozessor-Variable definiert, welche in der Datei `plasma_soc.h` ausgewertet wird und dadurch die passende Systemdefinition einbezogen wird. Die Headerverzeichnisse der Firmware werden ebenfalls umbenannt, um Kollisionen mit den Linux-Headerverzeichnissen zu vermeiden: *dev* und *sys* werden jeweils mit dem Präfix „k“ versehen.

Aus der Beispielanwendung werden die Funktionen für die UART-Ein- und Ausgabe sowie die Displayausgabe entfernt. Stattdessen erfolgt die Anzeige der seit dem Systemstart vergangenen Zeit über die auf dem Board vorhandenen 7-Segment Anzeigen. Dafür wird eine Funktion entwickelt, welche für Ganzzahlen im Bereich von 0x0...0xF für Common-Anode 7-Segment-Anzeigen geeignete Codes zurückgibt.

Die LED-Lauflichtfunktion wird ebenfalls erweitert, sodass die Richtung und Muster konfigurierbar sind. Das LED-Muster ist im externen SDRAM über die Adresse `0x800` und die Richtung über der Adresse `0x804` ansprechbar.

4.7. Einrichtung des Linux-Betriebssystems

4.7.1. Preloader



4.7.2. U-Boot




```
fatload mmc 0:${mmcloadpart} $fpgadata plasma_soc.rbf;
fpga load 0 $fpgadata $filesize;
run bridge_enable_handoff;

echo -- Setting Env Variables --
setenv fdtimage socfpga_cyclone5_de1_soc.dtb;
setenv mmcroot /dev/mmcblk0p2;
setenv mmcload 'mmc rescan;${mmcloadcmd} mmc 0:${mmcloadpart}
 ↪  ${loadaddr} ${bootimage};${mmcloadcmd} mmc 0:${mmcloadpart}
 ↪  ${fdtaddr} ${fdtimage};';
setenv mmcboot 'setenv bootargs earlyprintk debug console=ttyS0,115200
 ↪  root=${mmcroot} rootfstype=ext4 rw rootwait mem=768M; bootz
 ↪  ${loadaddr} - ${fdtaddr}';

run mmcload;
run mmcboot;
```

Listing 4.1: Boot-Script

4.7.3. Kernel



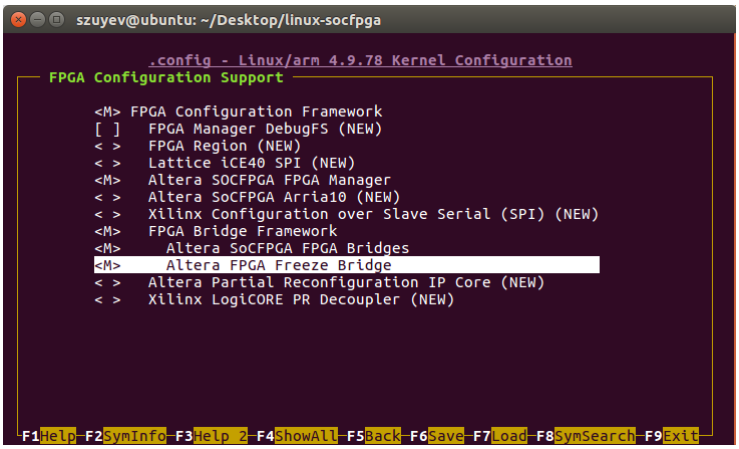


Abbildung 4.10.: Ausschnitt aus der Kernel-Konfiguration

4.7.4. Devicetree

Damit im Betrieb unter Linux die Hardware richtig erkannt wird und auch eigene Treiber geladen werden können, wird eine Devicetree-Definition erstellt.

Mit der Quartus Entwicklungsumgebung ist es möglich, mit dem in Listing 4.2 dargestellten Befehl, eine Device-Tree-Datei zu erstellen. Diese muss dann mit dem Devicetree-Compiler in ein vom Kernel verwertbares Binärformat konvertiert werden. Das `sopc2dts`-Programm erwartet eine `sopcinfo` Datei, welche bei der Generierung der HDL-Dateien im Platform Designer entsteht.

```
sopc2dts --input ${sopcinfo} \
--output dts/plasma_soc.dts \
--type dts \
--board dts/soc_system_board_info.xml \
--board dts/hps_clock_info.xml \
--bridge-removal all
```

Listing 4.2: Ausschnitt aus der generierten DTS-Datei

Die mit den Altera-Tools aus der `sopcinfo`-Datei generierte Devicetree-Defintion ist nicht zufriedenstellend. Bis auf den Plasma-SoC- und den SysID-Eintrag fehlen alle weiteren im Platform Designer angebundenen Komponenten. Außerdem sind einige Gerätebeschreibungen nicht mit dem Linux-Kernel 4.9.78 kompatibel, was zur Folge hat, dass die Treiber unter Umständen nicht korrekt funktioneren oder das System nicht startet.

```
...
hps_0_bridges: bridge@0xc0000000 {
        compatible = "altr,bridge-18.0", "simple-bus";
        reg = < 0xC0000000 0x20000000 0xFF200000 0x00200000 >;
        reg-names = "axi_h2f", "axi_h2f_lw";
        #address-cells = < 2 >;
        #size-cells = < 1 >;
        ranges = < 0x00000000 0x00000000 0xC0000000 0x20000000
         ↪  0x00000001 0x00000000 0xFF200000 0x00000008 >;

        plasma_soc_0: unknown@0x000000000 {
                compatible = "unknown,unknown-1.0";
                reg = < 0x00000000 0x00000000 0x20000000 >;
        }; //end unknown@0x000000000 (plasma_soc_0)

        sysid_qsys_0: sysid@0x100000000 {
                compatible = "ALTR,sysid-18.0", "ALTR,sysid-1.0",
                 ↪  "altr,sysid-1.0";
                reg = < 0x00000001 0x00000000 0x00000008 >;
                id = < 3735928559 >;        /*
                 ↪  embeddedsw.dts.params.id type NUMBER */
                timestamp = < 1536706156 >;        /*
                 ↪  embeddedsw.dts.params.timestamp type NUMBER */
        }; //end sysid@0x100000000 (sysid_qsys_0)
}; //end bridge@0xc0000000 (hps_0_bridges)
...
```

Listing 4.3: Ausschnitt aus der generierten DTS-Datei

Darum wird, aufbauend auf den Originaldateien im Kernel-Repository [22] und der Devicetree Spezifikation [23], eine eigene Devicetree-Definition für das Plasma-SoC auf dem DE1-SoC-Board entworfen. Die zugrundeliegenden Dateien aus dem Kernel-Repository sind *socfpga.dtsi* und *socfpga_cyclone5.dtsi* aus dem Verzeichnis `arch/arm/boot/dts`. Eine dritte Datei *socfpga_cyclone5_de1_soc.dts* wird erstellt, welche durch einen include-Mechanismus von den beiden vorher genannten Dateien erbt. Nachfolgend werden die wichtigsten Fragmente beschrieben.

Zunächst werden die Kernel-Bootargumente angegeben.

```
chosen {
        bootargs = "earlyprintk debug console=ttyS0,115200
         ↪  root=/dev/mmcblk0p2 rw rootwait mem=768M";
```

```
};.
```

Die Definition des Arbeitsspeichers wird um Einträge für den internen und externen (siehe Abschnitt 4.4.4) RAM des Plasma-SoC erweitert.

```
memory@0 {
        name = "memory";
        device_type = "memory";
        reg =   <0x00000000 0x40000000>, /* 1GB DDR3 */
                <0xC0000000 0x00002000>, /* 8KB Plasma RAM0 */
                <0xD0000000 0x00010000>; /* 64MB Plasma RAM1 */
};
```

Die aus Listing 4.3 bereits bekannte HPS-Bridge-Definition wird vervollständigt. Das Wichtigste hier ist die Adress-Translation über den `ranges`-Ausdruck: Die Basisadresse `0xFF200000` mit dem Adressbereich `0x8` wird auf `0,0`, die Basisadresse `0xC0000000` mit dem Adressbereich `0x5510` wird auf`1,0`, und die Basisadresse `0xD4000000` mit dem Adressbereich `0x80` wird auf `2,0` abgebildet.

```
hps_0_bridges: bridge@c0000000 {
        compatible = "altr,bridge-1.0", "simple-bus";
        reg = < 0xC0000000 0x20000000 0xFF200000 0x00200000 >;
        reg-names = "axi_h2f", "axi_h2f_lw";
        #address-cells = < 2 >;
        #size-cells = < 1 >;
        ranges = <0 0  0xFF200000 0x00000008 >,
                 <1 0  0xC0000000 0x00005510 >,
                 <2 0  0xD4000000 0x00000080 >;
         ...
```

Dem Adressbereich `0,0`, welcher aus der Basisadresse `0xFF200000` transliert wird, wird die Beschreibung des SysID-IP-Core zugeordnet.

```
hps_0_bridges: bridge@c0000000 {
...
        sysid_qsys_0: sysid@0,0 {
                compatible = "ALTR,sysid-13.1", "ALTR,sysid-1.0",
                 ↪  "altr,sysid-1.0";
                reg = <0 0 0x00000008>;
                id = < 3735928559 >;
                timestamp = < 1536342590 >;
        };
...
```

Weiter werden die Beschreibungen für die im Plasma-SoC am Wishbone-Bus angebundenen Timer, Zähler und GPIOs definiert. Diese sind dem aus der Basisadresse `0xC0000000` transliertem Adressbereich `1,0` zugeordnet.

```
hps_0_bridges: bridge@c0000000 {
...
        timer@1,5200 {
                compatible = "plsoc,timer";
                reg = <1 0x5200 0x10>;
        };

... weitere 3 timer

        counter@1,5300 {
                compatible = |\colorbox{orange}{"plsoc,counter"}|;
                reg = <1 0x5300 0x10>;
        };

... weitere 3 Zähler

        gpio@1,5400 {
                compatible = "plsoc,gpio";
                reg = <1 0x5400 0x10>;
        };

... weitere 3 GPIOs

        spi@1,5500 {
                compatible = "plsoc,spi";
                reg = <1 0x5500 0x10>;
        };
...
```

Zum Schluss werden die über die Avalon-Wishbone-Bridge angebundenen 6 7-Segment-Anzeigen, der Schalterblock und der Tasterblock beschrieben.

```
hps_0_bridges: bridge@c0000000 {
...
        hex0: gpio@2,70 {
                compatible = "altr,pio-15.0", "altr,pio-1.0";
                reg = <2 0x70 0x00000010>;
                altr,gpio-bank-width = <7>;
```

```
                #gpio-cells = <1>;
                gpio-controller;
        };

... weitere 5 7-Segment-Anzeigen

        switches: gpio@2,10 {
                compatible = "altr,pio-15.0", "altr,pio-1.0";
                reg = <2 0x10
                 ↪  0x00000010>;
                altr,gpio-bank-width = <10>;
                #gpio-cells = <1>;
                gpio-controller;
        };

        keys: gpio@2,0 {
                compatible = "altr,pio-15.0", "altr,pio-1.0";
                reg = <2 0x00 0x00000010>;
                altr,gpio-bank-width = <4>;
                #gpio-cells =
                 ↪  <1>;
                gpio-controller;
        };
...
```

4.7.5. Systemabbild

Es wird ein im Altera University Program verfügbares Systemabbild für das DE1-SoC-Board als Basis für eigene Anpassungen verwende. In diesem Abbild ist ein vorkonfiguriertes Ubuntu 12.04 Betriebssystem vorhanden. Der Vorteil des vorkonfigurierten Systemabbilds ist, dass damit ein geringer Aufwand nötig ist, um eine auf das DE1-SoC-Board mit einem Plasma-SoC zugeschnittene Betriebssystem-Distribution mit graphischer Oberfläche zu erstellen:

Quelle

Es werden der Preloader, der U-Boot Bootloader samt Konfiguration, der kompilierte Devicetree sowie der Kernel mit den Kernel-Modulen ersetzt durch Modifikationen bzw. Eigenentwicklungen aus Abschnitt 4.7.1, Abschnitt 4.7.2, Abschnitt 4.7.3 und Abschnitt 4.7.4.

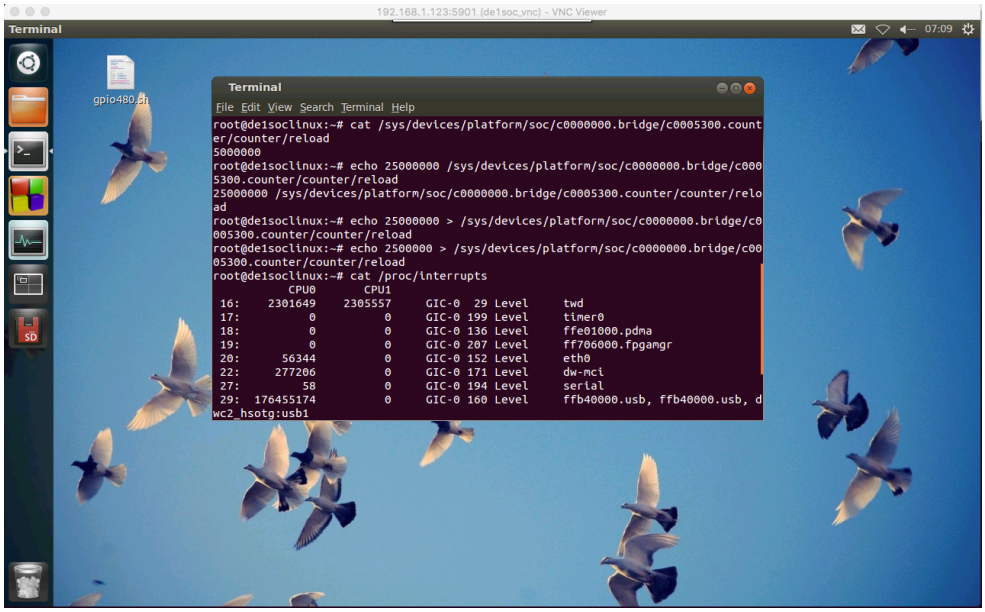


Abbildung 4.11.: Ubuntu 12.04-Distribution mit angepasstem Linux 4.9.78 Kernel

4.8. Entwicklung von Linux-Anwendungen für das HPS

Sysid

Update-LED-Pattern

4.9. Entwicklung von Linux-Treibern für das HPS

Es wird ein Treiber für den im Plasma-SoC entwickelten Zähler entwickelt.

Plasma-SoC Counter-Driver

```
...
static const struct of_device_id plsoc_counter_match[] = {
  { .compatible = "plsoc,counter" },
  { /* Sentinel */ }
};
...
```

```
...
ssize_t plsoc_counter_show_reload(struct device *dev, struct
↪  device_attribute *attr, char *buf)
```

```
{
  struct plsoc_counter *counter = dev_get_drvdata(dev);

  return scnprintf(buf, PAGE_SIZE, "%u\n", readl(counter->regs +
  ↪  PLSOC_REG_RELOAD));
}
...
```

```
...
ssize_t plsoc_counter_store_reload(struct device *dev, struct
↪  device_attribute *attr, const char *buf, size_t count)
{
  u32 reload;
  struct plsoc_counter *counter;

  if (buf == NULL) {
    pr_err("Error, string must not be NULL\n");
    return -EINVAL;
  }

  if (kstrtou32(buf, 10, &reload) < 0) {
    pr_err("Could not convert string to integer\n");
    return -EINVAL;
  }

  counter = dev_get_drvdata(dev);

  writel(reload, (counter->regs + PLSOC_REG_RELOAD));

  return count;
}
...
```

```
echo 2500000 > /sys/devices/platform/soc/c0000000.bridge/c0005300.counter/counter/
```

5. Ergebnisse und Ausblick

5.1. Zusammenfassung

5.2. Ausblick

Literatur

[1] S. Rhoads. (21. Nov. 2016). Plasma - most MIPS I(TM) opcodes, Adresse: https://opencores.org/project/plasma (besucht am 22. 04. 2018).

[2] B. Zeller, „SoC-based Image Classification using Binarized Neural Networks“, Aug. 2018.

[3] *ZedBoard, (Zynq™Evaluation and Development) Hardware User's Guide*, Version 2.2, AVNET, 27. Jan. 2014.

[4] *DE1-SoC, User Manual*, V2.0.3, Terasic, 24. Aug. 2016.

[5] P. P. Chu, „Embedded SoPC design with Nios II Processor and VHDL examples“, in. John Wiley & Sons, Inc, 2011, ISBN: 978-1-118-00888-1.

[6] M. Mitic, M. Stojcev und Z. Stamenkovic, „An Overview of SoC Buses“, 11. Juli 2007.

[7] W. D. P. et al. (2010). Wishbone B4, WISHBONE System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores, OpenCores, Adresse: https://cdn.opencores.org/downloads/wbspec_b4.pdf (besucht am 02. 04. 2018).

[8] *Avalon®Interface Specifications, Updated for Intel®Quartus®Prime Design Suite: 18.0*, MNL-AVABUSREF, Intel, 22. Mai 2018.

[9] P. Tofahrn und S. Zuyev, „Plasma, MIPS-Prozessor“, 1. Aug. 2017.

[10] boschmitt. (9. Okt. 2014). Wishbone, Wishbone Interconnection (INTERCON), Adresse: https://github.com/boschmitt/wishbone (besucht am 02. 04. 2018).

[11] P. Tofahrn, „Interrupt Controller, für die Plasma-CPU“, 30. Sep. 2017.

[12] R. Kim und M. Mattioli. (7. Mai 2017). ZedBoard OLED Display, Adresse: https://github.com/mmattioli/ZedBoard-OLED (besucht am 02. 04. 2018).

[13] S. et al. (25. Juli 2017). NexysVideo, Adresse: https://github.com/Digilent/NexysVideo/tree/master/Projects/GPIO/src/hdl (besucht am 02. 04. 2018).

[14] P. Johnston. (15. Feb. 2017). Implementing Malloc, First-fit Free List, Adresse: https://embeddedartistry.com/blog/2017/2/9/implementing-malloc-first-fit-free-list (besucht am 02. 04. 2018).

[15] *In-System Modification of Memory and Constants*, QII53012-12.0.0, Altera, Juni 2013.

[16] *Internal Memory (RAM and ROM), User Guide*, UG-01068-5.0, Altera, Juni 2014.

[17] X. Xing, C. Zezong, J. Jing und K. Hengyu, „Porting from Wishbone Bus to Avalon Bus in SoC Design“, in *2007 8th International Conference on Electronic Measurement and Instruments*, (16.–18. Aug. 2007), School of Electronic Information, Wuhan University, Wuhan 430079, China, IEEE, Okt. 2007. DOI: 10.1109/ICEMI.2007.4350589.

[18] B. Krill. (11. Okt. 2016). VHDL Round-Robin Arbiter, Adresse: https://www.krll.de/portfolio/vhdl-round-robin-arbiter (besucht am 27. 06. 2018).

[19] *IS42/45R86400D/16320D/32160D IS42/45S86400D/16320D/32160D, 16Mx32, 32Mx16, 64Mx8 512Mb SDRAM*, Rev. B, Integrated Silicon Solution, Inc., 12. Mai 2015.

[20] *Using the SDRAM on Altera's DE1-SoC Board with VHDL Designs, For Quartus II 14.0*, Altera, Aug. 2014.

[21] *Embedded Peripherals IP User Guide, Updated for Intel® Quartus® Prime Design Suite: 18.0*, UG-01085, Intel, 7. Mai 2018.

[22] (). Linux development repository for socfpga, Altera Opensource, Adresse: https://github.com/altera-opensource/linux-socfpga (besucht am 01. 09. 2018).

[23] *Devicetree Specification, Release v0.2*, devicetree.org, 20. Dez. 2017.

[24] *Product Specification, OEL Display Module*, UG-2832HSWEG04, SAS1-B020-B, Univision Technology Inc., 24. Juli 2009.

[25] S. Zuyev, „Plasma MIPS-Prozessor, Beispielanwendungen und Erweiterung des Prozessors“, 5. Okt. 2017.

[26] P. J. Ashenden und J. Lewis, „VHDL-2008, Just the New Stuff“, in. Morgan Kaufmann Publishers, 2008.

[27] R. Gutierrez-Osuna. (). Address decoding, Adresse: http://research.cs.tamu.edu/prism/lectures/mbsd/mbsd_l16.pdf (besucht am 02. 04. 2018).

A. Übersicht über elektronische Anhänge

Überarbeiten

- Bachelorarbeit in elektronischer Form
- Quellcode der Prototypen
  - *attachments/Plasma-SoC/*
- Synthetisierte *Bitstream*-Dateien
  - *attachments/bin/plasma_soc_top.bin*
  - *attachments/bin/plasma_soc_top_with_underflow_bug.bit*
- Datenblätter und Spezifikationen
  - *attachments/ZedBoard_HW_UG_v2_2.pdf* [3]
  - *attachments/wbspec_b4.pdf* [7]
  - *attachments/UG-2832HSWEG04.pdf* [24]
- Literatur und Dokumente
  - *attachments/PHV_Tofahrn* [11]
  - *attachments/plasma.pdf* [9]
  - *attachments/beispielanwendungen.pdf* [25]
  - *attachments/6195_C007.pdf* [6]
  - *attachments/Morgan Kaufmann VHDL-2008 Just the New Stuff-20147121184211.pdf* [26]

  - *attachments/mbsd_l16.pdf* [27]

- Webquellen
  - *attachments/archives/zedboard_oled.html* [12]
  - *attachments/archives/embeddedartistry_malloc.html* [14]
  - *attachments/archives/nexysvideo_oled.html* [13]
  - *attachments/archives/boschmitt_wishbone.html* [10]
  - *attachments/archives/rhoads_plasma.html* [1]